Introduction to the subject

After a detailed discussion on the digital communication basics, we now know that analog data can be converted to digital (binary) form and can be coded appropriately to be sent over communication channels. We also understood the need for better bandwidth efficiency and higher data rates of transmission. Thus we come to two very specific questions in communication theory. What is the ultimate level of data compression and what is the absolute maximum transmission rate? The study of information theory answers both these questions and also indicates methods of achieving these limits.

Due to its significant role in the field, information theory is widely considered a branch of communication theory. But in its true sense, the study of information theoretic concepts have applications in Mathematics, statistics, economics, computer science, physics, probability, etc. We, however, shall restrict our discussion on the use of information theory in the field of communication.

The foundations of Information theory are traced back to the an article in the Bell System Technical Journal, 1948 with the title "The Mathematical Theory of Communication" by the American Electrical engineer, Claude E. Shannon. This paper became the ground work for most of the modern day communication theory and information theory and thus Shannon is truly called the father of modern day Information theory.

Through this subject, we try to develop an understanding of the basic meaning of information in relation to communication theory, the method of data compression and limits on the same, and finally limits on the transmission media for optimum communication.

Let us start by developing an intuitive feel of what information really means.

Consider the following sentences:

a) Tomorrow, the sun will rise from the east.
b) The phone will ring in the next one hour.
c) It will snow in Delhi next winter.

Each of these represents different amounts of information. The first statement gives almost no information at all. This statement states something that is sure to happen and hence occurs with a probability of 1. The second statement gives us some more information. The phone may or may not ring in the next hour. The probability of the same will therefore be less than one. The third statement represents a very rare occurrence, meaning that the probability of its occurrence is very low. Therefore this statement gives us the largest amount of information. It is interesting to observe that as the probability of the occurrence of an even decreases, the amount of information that it reports increases. Also note that the amount of information in the statements has nothing to do with the length of the statements.

We can now develop a mathematical measure of information as suggested by the research done by Shannon.

Uncertainty and Information

Definition: Consider a discrete random variable X with the possible outcomes x_i , $i = 1, 2, 3,.., n$. The **self information** of the event $X = x_i$ is defined as

$$I(x_i) = \log_r\left(\frac{1}{P(x_i)}\right) = -\log(P(x_i))$$

We may note here that a high probability event conveys less information than a low probability event. For an event with $P(x_i) = 1$, $I(x_i) = 0$. Since a lower probability implies a higher degree of uncertainty (and vice versa), a random variable with a higher degree of uncertainty contains more information.

The above can be observed as a direct inference of the following properties of information.

1. Information conveyed by a message cannot be negative. It has to be at least 0.
2. If the event is definite, that is the probability of occurrence of the event is 1, then the information conveyed by the event is 0.
3. The information conveyed by a composite statement made by messages which are independent is simply given by the sum of the individual self-information contents. That is
$$I(m_1,m_2) = I(m_1) + I(m_2)$$

Further, with the knowledge of inverse proportionality of Information content with probability of even, the above properties combined give us the result that the logarithm function is the only form in which information content can be represented.

The units for $I(x_i)$ are determined by the base of the logarithm 'r'.

i) If $r = 2$, the unit is bits
ii) If $r = e$, the unit is Nats
iii) If $r = 10$, the unit is Hartley or Decits

Example 1: Consider a binary source which tosses a fair coin and outputs a 1 if a head appears and a 0 is a tail appears. For the source, P(1) = P(0) = 0.5. The information content of each output from the source is

$$I(x_i) = -\log_2 P(x_i) = -\log_2(0.5) = 1\ bit$$

Indeed, we have to use only one bit to represent the output from this binary source. Now suppose the successive outputs from this binary source are statistically independent, i.e., the source is memoryless. There are thus 2^m possible outcomes each of which is equiprobable with probability 2^{-m}. The self information of each possible outcome is

$$I(x_i) = -\log_2 P(x_i) = -\log_2(2^{-m}) = m\ bits$$

Again, we observe that, we indeed need m bits to represent the possible m outputs.

Example 2: Consider a source emitting two symbols s_0 and s_1 with the corresponding probabilities ¾ and ¼ respectively. Find the self-information of the symbols in

a) Bits
b) Decits
c) Nats

Solution:

a) We have

$$I(s_0) = log_2\left(\frac{1}{3/4}\right) = 0.415\ bits$$

$$I(s_1) = log_2\left(\frac{1}{1/4}\right) = 2\ bits$$

b) We have

$$I(s_0) = log_{10}\left(\frac{1}{3/4}\right) = 0.124\ decits$$

$$I(s_1) = log_{10}\left(\frac{1}{1/4}\right) = 0.602\ decits$$

c) We have

$$I(s_0) = log_e\left(\frac{1}{3/4}\right) = 0.287\ Nats$$

$$I(s_1) = log_e\left(\frac{1}{1/4}\right) = 1.386\ Nats$$

Average Information of a Zero memory source

A zero memory source or a discrete memoryless source is the one in which the emission of the current symbol is independent of the emission of the previous symbols. Consider a source emitting symbol S = {s_1, s_2, s_3,…,s_n} with respective probabilities P = {p_1, p_2, p_3,…, p_n}.

Now consider a long message of length 'L' emitted by the source. Then it contains

p_1L number of symbols of 's_1',

p_2L number of symbols of 's_2',

…

…

p_nL number of symbols of 's_n'

The self information of each s_i is given by

$$I(s_i) = log_2 \frac{1}{p_i} \; bits$$

On an average, in a long sequence of length L, each symbol will occur p_iL number of times. Hence, total information conveyed by any particular symbol s_i will be

$$p_i L \; log_2 \left(\frac{1}{p_i}\right) bits$$

Therefore, the total information conveyed by the source is simply the sum of all these information contents, that is,

$$I(S) = p_1 L \; log_2 \left(\frac{1}{p_1}\right) + p_2 L \; log_2 \left(\frac{1}{p_2}\right) + \cdots + p_n L \; log_2 \left(\frac{1}{p_n}\right)$$

Thus, the average information conveyed by the source by emitting 'L' symbols is denoted by its entropy H(S), which is given by the expression

$$H(S) = \frac{I(S)}{L} = p_1 \; log_2 \left(\frac{1}{p_1}\right) + p_2 \; log_2 \left(\frac{1}{p_2}\right) + \cdots + p_n \; log_2 \left(\frac{1}{p_n}\right)$$

We can simply rewrite this as

$$H(S) = \sum_{i=1}^{n} p_i \; log_2 \left(\frac{1}{p_i}\right) \; bits/sym$$

Hence, H(S) or entropy gives the measure of the average information content of the symbols of a source S.

For a Binary memoryless source, the entropy would hence be

$$H(S) = p \log \frac{1}{p} + (1-p) \log \frac{1}{1-p}$$

Where p represents the probability of occurrence of one of the symbols and 1-p represents the probability of the other symbol.

The average rate of transmission can also be defined for an information system if the symbol rate or baud rate of the system is known. If the baud rate of the system is r_S sym/s, then the average rate of information is given by

$$R_S = H(S) * r_s \text{ bits/s}$$

Note: The entropy of X can also be interpreted as the expected value of the random variable log(1/p(X)), where X is a random variable with probability mass function (discrete form of probability distribution function) p(X).

Note: By definition of entropy, we can change its unit by using the formula:

$$H_b(X) = (\log_b a)\, H_a(X)$$

Example 3: A discrete memoryless source emits one of the five possible symbols every second. The symbol probabilities are {1/4, 1/8, 1/8, 3/16, 5/16}. Find the average information content of the source in bits/sym, nats/sym and Hartley/sym.

Solution:

Average information content or Entropy of this source can be given by

$$H_2(S) = \sum_{i=1}^{5} p_i \, log_2\left(\frac{1}{p_i}\right) \; bits/sym$$

$$= -\left(\frac{1}{4}\log_2\frac{1}{4} + \frac{1}{8}\log_2\frac{1}{8} + \frac{1}{8}\log_2\frac{1}{8} + \frac{3}{16}\log_2\frac{3}{16} + \frac{5}{16}\log_2\frac{5}{16}\right) = 2.227 \; bits/sym$$

Also

$$H_e(S) = \; H_2(S)\log_e 2 = 2.227 \times 0.693 = 1.543 \; nats/sym$$

and

$$H_{10}(S) = \; H_2(S)\log_{10} 2 = 2.227 \times 0.3010 = 0.6704 \; Hartley/sym$$

Example 4: The international Morse code uses a sequence of symbols of dots and dashes to transmit letters of the English alphabet. The dash is represented by a current pulse of duration 2 ms and dot by duration of 1 ms. The probability of dash is half as that of dot. Consider 1ms duration of gap is given in between the symbols. Calculate

a) Self information of a dot and a dash
b) Average information content of a dot-dash code
c) Average rate of information

Solution

a) Let p_{dot} and p_{dash} be the probabilities of dot and dash, respectively. Given

$$p_{dash} = \frac{1}{2}p_{dot}$$

Also $p_{dot} + p_{dash} = 1$. Therefore,

$$p_{dot} + \frac{1}{2}p_{dot} = 1 \Rightarrow p_{dot} = \frac{2}{3}$$

We can also deduce that

$$p_{dash} = \frac{1}{2} p_{dot} = \frac{1}{3}$$

Now,

$$I(dot) = -\log_2 p_{dot} = -\log_2 \frac{2}{3} = 0.5849\ bits$$

$$I(dash) = -\log_2 p_{dash} = -\log_2 \frac{1}{3} = 1.5849\ bits$$

b) $H(S) = -(p_{dot} \log_2 p_{dot} + p_{dash} \log_2 p_{dash}) = 0.9182\ bits/sym$

c) From the probabilities of dot and dash, it is clear that for every three symbols transmitted, there will be one symbol of type dash and two of type dot. Also the duration of dash is 2ms and that of dot is 1ms, and the 1ms gap is left in between the symbols. Therefore a total of (1+1+1+1+2+1) = 7ms time is required to transmit an average of 3 symbols (· · −). So the symbol rate is given by

$$r_s = \frac{\frac{3}{7} sym}{ms} = \frac{\frac{3000}{7} sym}{s}$$

$$\Rightarrow R_s = H(S) \times r_s = 0.9182 \times \frac{3000}{7} = 393.51\ bits/s$$

Properties of Entropy:

1. Entropy is a continuous function of probability.
2. Entropy of a system is the same irrespective of the order in which the symbols are arranged.
3. Entropy is never negative. It can take a minimum value of 0 if the event in question is certain or deterministic.
4. Entropy has maximum value when all the possible outcomes of an event are equiprobable. (Proof attached separately, Sheet 1)

Source Efficiency and Source Redundancy

Source Efficiency is defined as the ratio of the average information conveyed by the source to that of the maximum average information

$$\eta_S = \frac{H(S)}{H(S)_{max}} \times 100\%$$

Source redundancy can be defined

$$R_{\eta S} = \left[1 - \frac{H(S)}{H(S)_{max}}\right] \times 100\% = \frac{H(S)_{max} - H(S)}{H(S)_{max}} \times 100\%$$

Example 5: Consider a system emitting one of the three symbols A, B and C, with respective probabilities 0.6, 0.25 and 0.15. Calculate its efficiency and redundancy.

Solution: The efficiency is given by

$$\eta_S = \frac{H(S)}{H(S)_{max}} \times 100\%$$

Now

$$H(S) = 0.6 \log_2 \frac{1}{0.6} + 0.25 \log_2 \frac{1}{0.25} + 0.15 \log_2 \frac{1}{0.15} = 1.3527 \; bits/sym$$

And

$$H(S)_{max} = \log_2 3 = 1.5849 \; bits/sym$$

The efficiency is hence given by

$$\eta_S = \frac{1.3527}{1.5849} \times 100\% = 85.35\%$$

And the redundancy is

$$R_{\eta S} = \frac{1.5849 - 1.3527}{1.5849} \times 100\% = 14.65\%$$

Mutual Information

Consider two discrete random variables X and Y with possible outcomes x_i, i = 1, 2, 3, .. , n, and y_j, j = 1, 2, 3, .., m respectively. Suppose we observe some outcome Y = y_j and we want to determine the amount of information this event provides about the event X=x_i. We may note that this information will have to satisfy the following extreme case conditions.

1. If X and Y are independent, occurrence of Y=y_j provides no information about X=x_i.
2. If X and Y are fully dependent events, in which case the occurrence of Y = y_j determines the occurrence of the event X=x_i.

A suitable measure that satisfies these conditions is the logarithm of the ratio of the conditional probability

$$P\big(X = x_i \big| Y = y_j\big) = P(x_i|y_j)$$

divided by the probability

$$P(X = x_i) = P(x_i)$$

The mutual information, I(x_i; y_j) between x_i and y_j, can thus be defined as

$$I(x_i; y_j) = \log_2\left(\frac{P(x_i|y_j)}{P(x_i)}\right)$$

As before, the units of I(x) are determined by the base of the logarithm, which is usually selected as 2 for which the units are Bits. Note that

$$\frac{P(x_i|y_j)}{P(x_i)} = \frac{P(x_i|y_j)P(y_j)}{P(x_i)P(y_j)} = \frac{P(x_i; y_j)}{P(x_i)P(y_j)} = \frac{P(y_j|x_i)}{P(y_j)}$$

by Baye's Theorem

Therefore,

$$I(x_i; y_j) = \log\left(\frac{P(x_i|y_j)}{P(x_i)}\right) = \log\left(\frac{P(y_j|x_i)}{P(y_j)}\right) = \log\left(\frac{P(x_i; y_j)}{P(x_i)P(y_j)}\right) = I(y_j; x_i)$$

Physical interpretation of $I(x_i; y_j) = I(y_j; x_i)$ is that the information provided by the occurrence of the event $Y=y_j$ about the event $X=x_i$ is identical to the information provided by the occurrence of the event $X=x_i$ about the event $Y=y_j$.

Let us now verify the two extreme cases:

1. When the random variables X and Y are statistically independent, $P(x_i \mid y_j) = P(x_i)$, which leads to $I(x_i; y_j) = 0$.
2. When the occurrence of $Y=y_j$ uniquely determines the occurrence of the event $X=x_i$, $P(x_i \mid y_j) = 1$, and the mutual information becomes

$$I(x_i; y_j) = \log\left(\frac{1}{P(x_i)}\right) = -\log P(x_i).$$

This is the self information of the event $X=x_i$.

Joint Entropy

The joint entropy H(X, Y) of a pair of discrete random variables (X, Y) with a joint distribution p(x, y) is defined as

$$H(X,Y) = -\sum_{x\in X}\sum_{y\in Y} p(x,y)\log_2 p(x,y),$$

which can also be expressed as

$$H(X,Y) = -E\{\log p(X,Y)\}$$

i.e. the expected value of log(p(X,Y)).

Conditional Entropy

If (X, Y) ~ p(x, y), the conditional entropy H(Y|X), i.e. the entropy of Y given the value of X, is defined as

$$H(Y|X) = \sum_{x \in X} p(x) H(Y|X = x)$$

$$= -\sum_{x \in X} p(x) \sum_{y \in Y} p(y|x) \log p(y|x)$$

$$= -\sum_{x \in X} \sum_{y \in Y} p(x, y) \log p(y|x)$$

$$= -E\{\log p(Y|X)\}$$

The naturalness of the definition of joint entropy and conditional entropy is exhibited by the fact that the entropy of a pair of random variables is the entropy of one plus the conditional entropy of the other. This can be proved by the following theorem called the Chain Rule of Entropy.

$$H(X, Y) = H(X) + H(Y|X)$$

Proof:

$$H(X, Y) = -\sum_{x \in X} \sum_{y \in Y} p(x, y) \log p(x, y)$$

$$= -\sum_{x \in X} \sum_{y \in Y} p(x, y) \log(p(x) p(y|x))$$

$$= -\sum_{x \in X} \sum_{y \in Y} p(x, y) \log p(x) - \sum_{x \in X} \sum_{y \in Y} p(x, y) \log p(y|x)$$

$$= -\sum_{x \in X} p(x) \log p(x) - \sum_{x \in X} \sum_{y \in Y} p(x, y) \log p(y|x)$$

$$= H(X) + H(Y|X)$$

The same can be extended for n random variables. Let X_1, X_2, X_3, … , X_n be jointly distributed with density function $p(x_1, x_2, x_3, \dots, x_n)$. Then,

$$H(X_1, X_2, \cdots, X_n) = H(X_1) + H(X_2|X_1) + \cdots + H(X_n|X_{n-1} \cdots X_1)$$

$$= \sum_{i=1}^{n} H(X_i|X_{i-1} \cdots X_1)$$

Average Mutual Information and Entropy

Average mutual information can be calculated just as the average self information.

$$I(X,Y) = \sum_{x\in X}\sum_{y\in Y} P(x_i,y_j)I(x_i,y_j)$$

$$= \sum_{x\in X}\sum_{y\in Y} P(x_i,y_j)\log\left(\frac{P(x_i|y_j)}{P(x_i)}\right)$$

$$= -\sum_{x\in X}\sum_{y\in Y} P(x_i,y_j)\log P(x_i) - \left(-\sum_{x\in X}\sum_{y\in Y} P(x_i,y_j)\log\big(P(x_i|y_i)\big)\right)$$

$$= -\sum_{x\in X} P(x_i)\log P(x_i) - H(X|Y) = H(X) - H(X|Y)$$

Thus, physically, it may be interpreted that average mutual information is the reduction of uncertainty of X due to the knowledge of Y.

By symmetry, it also follows that

$$I(X,Y) = H(Y) - H(Y|X)$$

This means that X says as much about Y as Y says about X.

Since by Chain rule we know that

$$H(X,Y) = H(X) + H(Y|X)$$

We may rewrite the expression for average mutual information as

$$I(X,Y) = H(X) + H(Y) - H(X,Y).$$

We may also note that,

$$I(X,X) = H(X) - H(X|X) = H(X)$$

Thus the average mutual information of a random variable with itself is the entropy of the random variable. This is also a testament to the fact that entropy is referred to as Average Self Information. Hence in summary, we may collect the following results and follow them as a theorem.

$$I(X,Y) = H(X) - H(X|Y)$$

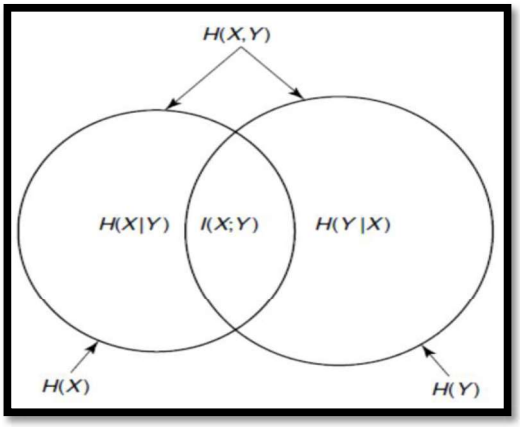
$$I(X,Y) = H(Y) - H(Y|X)$$

$$I(X,Y) = H(X) + H(Y) - H(X,Y)$$

$$I(X,Y) = I(Y,X)$$

$$I(X,X) = H(X)$$

The following Venn Diagram further explains the relationship between Conditional entropy, joint entropy, entropy and average mutual information of two random variables.



Extension of a zero memory source

Consider a zero memory source emitting two symbols 's_1' and 's_2' with probabilities 'p_1' and 'p_2' respectively. Obviously, $p_1+p_2=1$. Also the entropy of the source S is given by

$$H(S) = p_1 \log\frac{1}{p_1} + p_2 \log\frac{1}{p_2}$$

Now consider the second order extension of the source S. This source is denoted as S^2. Now the source S^2 will have four combinations viz. s_1s_1, s_1s_2, s_2s_1 and s2s2. Since in a zero memory source, the symbols are statistically independent, corresponding probabilities are given by

$$P(s_1s_1) = p(s_1)p(s_1) = p_1^2$$

$$P(s_1s_2) = p(s_1)p(s_2) = p_1p_2$$

$$P(s_2s_1) = p(s_2)p(s_1) = p_1p_2$$

$$P(s_2s_2) = p(s_2)p(s_2) = p_2^2$$

The entropy of the second order extension of the source is given by

$$H(S^2) = p_1^2 \log\left(\frac{1}{p_1^2}\right) + p_1p_2 \log\left(\frac{1}{p_1p_2}\right) + p_2p_1 \log\left(\frac{1}{p_2p_1}\right) + p_2^2 \log\left(\frac{1}{p_2^2}\right)$$

Simplifying and using the fact that $p_1+p_2=1$, one may simplify this to be

$$H(S^2) = 2\left(p_1 \log\frac{1}{p_1} + p_2 \log\frac{1}{p_2}\right)$$

Which simply means that

$$H(S^2) = 2 \cdot H(S)$$

In general, it can be observed in a similar fashion that for an n-th order extension of S,

$$H(S^n) = n \cdot H(S)$$

Source Coding Theorem

An important problem in communications is the efficient representation of data generated by a discrete source. The process by which this representation is accomplished is called source encoding. For efficient source coding, we need the statistical knowledge of the source. The primary motivation is the compression of data due to efficient representation of the symbols.

Suppose a discrete memoryless source outputs a symbol every t seconds. Each symbol is selected from a finite set of symbols x_i, i=1, 2, …, L, occurring with probabilities $P(x_i)$, i=1, 2, …, L. The entropy of this DMS in bits per symbol is given by

$$H(X) = \sum_{i=1}^{L} P(x_i) \log_2 \frac{1}{P(x_i)} \leq \log_2 L$$

where the equality holds when the symbols are equally likely. It implies that the average number of bits per source symbol is H(X) and the source rate is H(X)/t bits/sec. Now let us suppose that we wish to represent the 26 letters in the English alphabet using bits. We observe that 2^5=32>26. Hence each of the letters can be uniquely represented using 5 bits. This is an example of a Fixed Length Code (FLC). Each letter has a corresponding 5 bits long codeword.

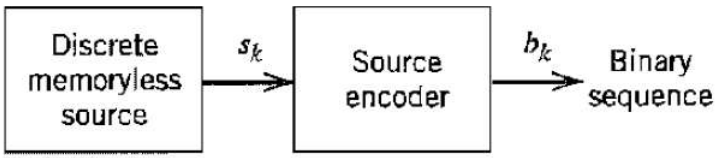
Note: A code is a set of vectors called codewords.

The fixed length code of the English alphabet is useful only if we take the assumption that each of the letters in the alphabet are equally probable to occur. However, we know that some of the letters are less common (x, q, j, z, etc.) while some others are more frequently used (e, s, t, etc.). It appears that allotting equal number of bits to both the frequently used letters as well as not so commonly used letters is NOT an efficient way of representation (coding), Intuitively, we should represent the more frequently occurring letters by a fewer number of bits and represent the more frequently occurring letters by larger number of bits. In this manner, if we have to encode a whole page of written text, we might end up using fewer number of bits overall. Hence, when the source symbols are not equally probable, a more efficient method to use is a Variable Length Code (VLC). The Morse code is an example of a VLC. In the Morse code, the letters of the alphabet and the numerals are encoded into streams of marks and spaces, denoted as dots "." and dashes "–", respectively. In the English language the letter E occurs most often while the letter Q occurs least often. The Morse code, hence allocates the shortest code, i.e. a single dot "." as codeword for the letter E, and encodes Q with the longest codeword in the code "– – . – ".

Our primary interest is in the development of an efficient source encoder that satisfies two functional requirements:

1. The codewords produced by the encoder are in binary form.
2. The source code in uniquely decodable, so that the original sequence can be reconstructed perfectly from the encoded binary sequence.

Consider the scheme given below.



We assume that the source has an alphabet with K different symbols and that the kth symbol s_k has the probability p_k, k=1, 2, 3, … , K. Let the binary codeword assigned to the symbol s_k be of length given by l_k measured in bits. We define the average codeword length, **L**, of the source encoder as

$$\boldsymbol{L} = \sum_{i=1}^{k} p_i l_i$$

In physical terms, **L** represents the average number of bits per source symbol used in the source encoding process. Let $\mathbf{L_{min}}$ define the minimum possible value of **L**. We then may define the coding efficiency of the source encoder as

$$\eta_C = \frac{\boldsymbol{L_{min}}}{\boldsymbol{L}}$$

The efficiency of this model, hence, increases when **L** reaches $\mathbf{L_{min}}$**.** A source encoder is hence called efficient when η approaches unity.

But how do we evaluate the minimum value of **L** i.e. $\mathbf{L_{min}}$?

The answer to this fundamental question is answered by Shannon's First Theorem: *The Source-coding Theorem*, which may be stated as follows:

Given a discrete memoryless source S, of entropy H(S) the average codeword length **L** for any distortionless source encoding scheme is bounded as

$$\boldsymbol{L} \geq H(S)$$

It is intuitively apparent as well that the above condition will be true as the entropy represents the average minimum number of bits required to represent the various symbols at the output of the source which will always be less than the actual average of the number of bits used to represent these symbols.

The efficiency of the source encoder is hence given by

$$\eta_C = \frac{H(S)}{\boldsymbol{L}} \times 100\%$$

The redundancy of the implemented code is given by

The redundancy of the code is given by

$$R_{\eta C} = (1 - \eta_C)$$

Data Compaction and Types of Source Codes

A common characteristic of signals generated by physical sources is that, in their natural form, they contain a significant amount of information that is redundant, the transmission of which is therefore wasteful of primary communication resources. For efficient signal transmission, the *redundant information should be removed from the signal prior to transmission*. This operation, with no loss of information, is ordinarily performed on a signal in digital from, in which case we refer to it as *data compaction* or *lossless data compression*. The code resulting from such an operation provides a representation of the source output that is not only efficient in terms of the average number of bits per symbol but also exact in the sense that the original data can be reconstructed with no loss of information. The entropy of the source establishes the fundamental limit on the removal of redundancy from the data. Applications such as military or medical information systems demand utmost reliability and hence the compression has to be lossless.

Some applications such as video conferencing and personal communication can tolerate certain degree of loss in the information, in which case, a higher compression ratio can be achieved, however at the cost of degradation of the reliability of the system. Lossy compression algorithms are used for such applications.

Most of the codes realized in real time are variable length codes. Let us discuss different types of codes.

a) Block codes

In this type, each symbol will be mapped onto a block of code symbols defined in the code alphabet. The block codes can be of either fixed or variable length. For example:

Symbol	Codeword 1	Codeword 2
A	00	1
B	01	01
C	10	110
D	11	111

b) Non-singular Codes

A block code is said to be non-singular if all its codewords are distinct. For example:

Symbol	Codeword 1	Codeword 2
A	00	1
B	01	01
C	10	110
D	11	111
	Non-Singular	Singular

c) Uniquely Decodable Codes

A non singular block code is said to be uniquely decodable if its *n*th extension is also non-singular for all finite values of '*n*'.

Symbol	Codeword 1	Codeword 2
A	00	1
B	01	01
C	10	110
D	11	111

d) Instantaneous codes

A uniquely decodable code is said to be an instantaneous code if the end of any codeword can be determined without the interpretation of the succeeding symbol.

It must be understood here how uniquely decodable codes are different from instantaneous code. The requirement is that the code should be able to decode the information without any ambiguity or need of additional information. Consider the following uniquely decodable code.

Symbol	Codeword 1
A	0
B	01
C	11
D	10

Let the transmitted information be 'ABCAD' and hence, the corresponding codeword transmitted is '00111010'. The receiver decodes it as 'A' because '0' is a valid codeword for 'A'. The next bit received is also '0'. Although the transmitted codeword was '01' corresponding to symbol 'B', the receiver incorrectly decodes it as 'A' because '0' itself is a valid codeword. Next bit that it receives is '1' that is not a valid codeword and, hence, it accepts the next bit getting '11' which is a valid codeword and, hence, it decoded by the receiver will be 'AACDD', which is not the same as that of the transmitted information. Thus, the codewords assigned for the symbols are not instantaneous.

The problem with the decoding in the previous scenario is that the codeword for B has started with symbol '0' that itself is a valid codeword. Thus, the receiver has incorrectly decoded the information as soon as the first symbol of the second codeword is received. From the above scenario, we can conclude that the decoding would be unsuccessful if any of the codewords start with any of the valid codewords of the system. In other words, the necessary and sufficient condition for a uniquely decodable code to be instantaneous is that *no codeword should be a prefix of any other codeword.* Thus they are also called *prefix codes*.

It can be proven that all prefix codes must satisfy the condition

$$\sum_{k=1}^{N} r^{-l_k} \leq 1$$

where 'r' is the number of symbols in code alphabet, that is r=2 for binary, 3 for ternary, etc. This is known as the Kraft McMillan Inequality. **Please refer to Sheet 2 for proof.**

If the Kraft McMillan inequality is not satisfied, we cannot construct a prefix code for a given set of codeword lengths. However, the satisfaction of the inequality does not imply that the corresponding codewords are prefix. In other words, *all prefix codes satisfy Kraft McMillan Inequality, but all codes satisfying the inequality need not be prefix.*

e) Optimal Codes

Instantaneous codes are called optimal if the lengths of the codewords assigned to the symbols are of minimum length.

Basically, it can be proven that for optimal codes the average length of the codeword should lie in the range

$$H(S) \leq L \leq H(S) + 1$$

Example 6: Following table gives different codes for a set of five symbols. Determine which of the following are valid prefix codes.

Code A	Code B	Code C	Code D
0	1	00	10
10	01	110	111
110	111	1110	110
1110	10	001	01
111	00	011	00

Solution

Code A: It is not a prefix code as the codeword '111' is a prefix of the codeword '1110'

Code B: It is not a prefix code as the codeowrd '1' is a prefix of the codewords '111' and '10'

Code C: It is not a prefix code as the codeword '00' is a prefix of the codeword '001'

Code D: It is a valid prefix code since no codeword is a prefix of other codewords.

Example 7: Which of the sets of the following requirements shown in the table below can be considered for the construction of the binary prefix codes?

Code P	Code Q	Code R	Word Length
1	2	1	1
1	1	1	2
2	2	1	3
2	1	2	4

Solution: Any set of lengths of the codewords that satisfy the Kraft McMillan Inequality can be considered for the construction of the prefix codes.

Code P: 1 code word of length 1, 1 code word of length 2, 2 code words of length 3, 2 code words of length 4.

$$\sum_{i=1}^{6} 2^{-l_i} = (1 \times 2^{-1}) + (1 \times 2^{-2}) + (2 \times 2^{-3}) + (2 \times 2^{-4}) = \frac{9}{8} > 1$$

Hence the code P cannot be a valid prefix code.

Code Q: Using similar method

$$\sum_{i=1}^{6} 2^{-l_i} = (2 \times 2^{-1}) + (1 \times 2^{-2}) + (2 \times 2^{-3}) + (1 \times 2^{-4}) = \frac{25}{16} > 1$$

Hence Q can also not be valid prefix code

Code R: For this code

$$\sum_{i=1}^{5} 2^{-l_i} = (1 \times 2^{-1}) + (1 \times 2^{-2}) + (1 \times 2^{-3}) + (2 \times 2^{-4}) = 1$$

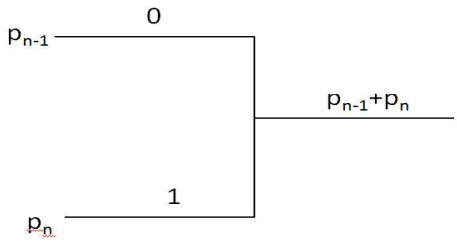
Therefore, R can be made into a valid prefix code. Possible set of codewords are as follows:

1	0
01	10
001	110
0001	1110

Huffman Code

Huffman Code is one of the pioneer works in variable length coding and gives an optimum source code representation for a discrete memoryless source with the source symbols that are not equally probable. The steps of the Huffman coding algorithm are as given below.

1. Arrange the source symbols in a non-increasing order of their probabilities.
2. Take the bottom two symbols and tie them together as shown in figure below. Add the probabilities of the two symbols and write it on the combined node. Label the two branches with a '1' and a '0' as depicted in the figure.



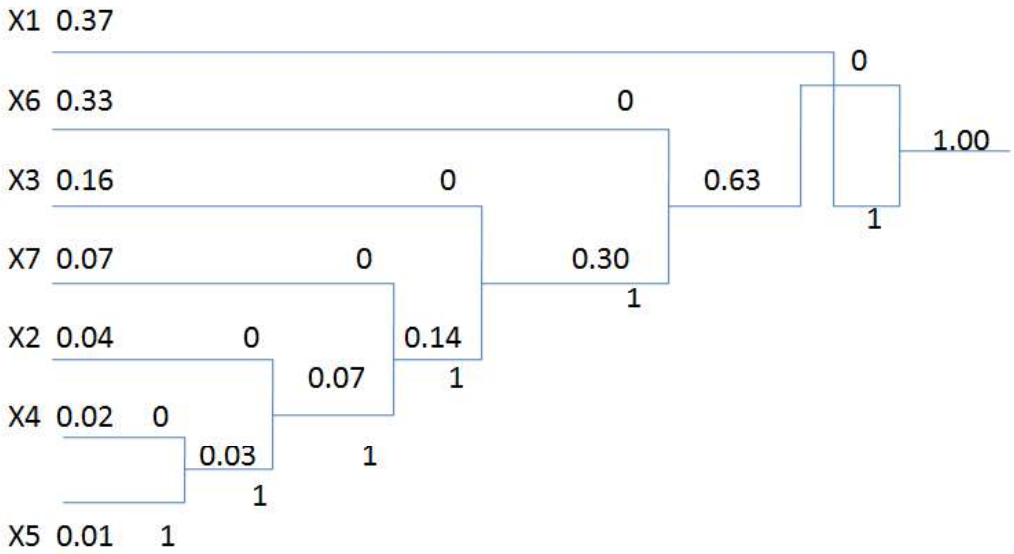
3. Treat the sum of probabilities as a new probability associated with a new symbol. Again pick the two smallest probabilities, tie them together to form a new probability. Each time we perform a combination of two symbols we reduce the total number of symbols by one. Whenever we tie together two probabilities (nodes), we label the two branches with a '1' and a '0'.
4. Continue this procedure until only one probability is left which has to be 1. This completes the construction of what we call the Huffman Tree.
5. To find out the prefix codeword for any symbol, follow the branches from the final node back to the symbol. While tracing back the route, read out the labels on the branches. This is the codeword for the symbol.

Example 8: Consider a DMS with seven possible symbols x_i, i=1,2,…,7 and the corresponding probabilities $P(x_i)$ ={0.37, 0.04, 0.16, 0.02, 0.01, 0.33, 0.07}. Construct the Huffman Tree for the given example and find the prefix code for the same.

Solution:

We first arrange the probabilities in decreasing order and then construct the Huffman tree.

Symbol	Probability	Self-Information	Codeword
X1	0.37	1.4344	1
X6	0.33	1.5995	00
X3	0.16	2.6439	010
X7	0.07	3.8365	0110
X2	0.04	4.6439	01110
X4	0.02	5.6439	011110
X5	0.01	6.6439	011111



We can find the entropy of the source to be

$$H(S) = -\sum_{k=1}^{7} P(x_k) \log_2 P(x_k) = 2.1152 \; bits$$

and the average number of binary digits needed per symbol is calculated to be

$$L = \sum_{i=1}^{7} p_i l_i = 1(0.37) + 2(0.33) + 3(0.16) + 4(0.07) + 5(0.04) + 6(0.02) + 6(0.01)$$
$$= 2.1700 \; bits$$

Efficiency of this code is

$$\eta_C = \frac{H(S)}{\boldsymbol{L}} \times 100\% = \frac{2.1152}{2.1700} \times 100\% = 97.47\%$$

Practice more problems for Huffman code and also note that the code is not unique. At any stage of the code, in case we have a condition where two symbols have same probability, then either can be taken on the upper branch hence leading to different set of codewords. But even in this condition, the average number of bits of the code is not going to be affected.

Practice Problems:

Repeat Example 8 for the following set of probabilities.

a) {0.46, 0.02, 0.01, 0.03, 0.06, 0.30, 0.12}
b) {1/2, $1/2^2$,$1/2^3$, $1/2^4$, $1/2^5$, $1/2^6$, $1/2^6$}
c) {1/8, 1/8, 1/8, 1/8, 1/8, 1/8, 1/8, 1/8}

Channel Coding

Channel coding refers to the class of signal transformations designed to improve communication performance by enabling the transmitted signals to better withstand the effects of various channel impairments such as noise, interference and fading.

Broadly, channel coding can be classified into 2 areas of study.

a) Waveform Coding
b) Structured Sequences

Waveform coding deals with transforming the symbol waveforms into "better waveforms" so as to make their detection more immune to errors. It involves coding the existing signal waveforms to form orthogonal or bi-orthogonal systems. For an M-ary signal, it can be seen that the probability of bit error reduces with increase in M if the waveforms are orthogonally or bi-orthogonally coded. The problem with this system is that increasing M also increases the bandwidth requirement of the system.

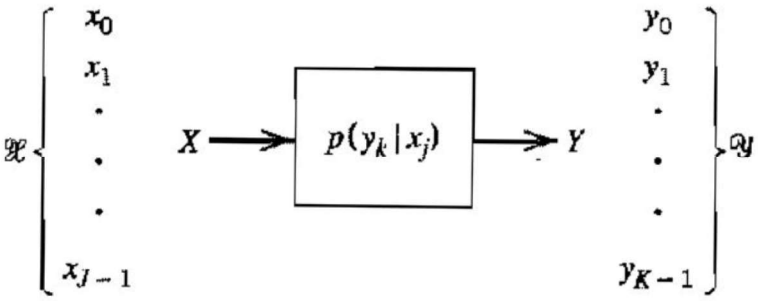
Structured sequences deal with transforming data sequences themselves into "better sequences" having structured redundancies or redundancy bits. The redundant bits can then be used for detection and correction of errors. Study of structured sequences is classified into 3 major types: Block coding, Convolution Coding and Turbo Coding.

The aim of our discussion is to develop and understanding of all these three types of coding techniques. But before a detailed discussion on channel coding can commence, it is important to understand the concept of channel models and channel capacity.

Discrete Memoryless Channels

Analogous to the concept of memoryless systems, i.e. a system in which the output of the system depends only on the current input of the system and not on past or future values, a memoryless channel in one in which every output sequence, say $\underline{Y} = [Y_1, Y_2, \ldots, Y_N]^T$ depends only on its corresponding input sequence say $\underline{X} = [X_1, X_2, \ldots, X_N]^T$. This means that the different outputs observed and different time instants are independent of each other. In case of a channel that has memory, the current output would be dependent on outputs of other time instants as well. Now, since the codeword alphabet is discrete in nature which means there are a discrete set of possible inputs, such channels are called Discrete Memoryless Channels.

A discrete memoryless channel would thus be a statistical model with an input X and an output Y that is a noisy version of X; both X and Y are random variables. Every unit of time, the channel accepts an input symbol X selected from an input alphabet and, in response, it emits an output symbol Y from the output alphabet. The channel is said to be discrete when both of the alphabets have finite sizes. It is said to be memoryless when the current output depends only on the current input symbol and not any of the previous ones.



The figure above shows a discrete memoryless channel described in terms of an input alphabet

$$\mathbf{X}=\{x_0, x_1, x_2, \ldots, x_{J-1}\},$$

an output alphabet,

$$\mathbf{Y}=\{y_0, y_1, y_2, \ldots, y_{K-1}\}$$

and a set of state transition probabilities

$$p(y_k|x_j) = P(Y=y_k \mid X=x_j) \qquad \text{for all j and k}$$

Note that the input alphabet and output alphabet may be different in size. A convenient way of describing a discrete memoryless channel is to arrange the various transition probabilities of the channel in the form of a matrix as follows:

$$P = \begin{bmatrix} p(y_0|x_0) & p(y_1|x_0) & \cdots & p(y_{k-1}|x_0) \\ p(y_0|x_1) & p(y_1|x_1) & & p(y_{k-1}|x_1) \\ \vdots & & \ddots & \vdots \\ p(y_0|x_{j-1}) & p(y_1|x_{j-1}) & \cdots & p(y_{k-1}|x_{k-1}) \end{bmatrix}$$

The J-by-K matrix P is called the channel matrix, or transition matrix. Note that each row of the channel matrix P corresponds to a fixed channel input, whereas each column of the matrix corresponds to a fixed channel output. Note also that a fundamental property of the channel matrix P, as defined here, is that the sum of the elements along any row of the matrix is always equal to one; that is,

$$\sum_{k=0}^{K-1} p(y_k|x_j) = 1 \qquad for\ all\ j$$

Suppose now that the inputs to a discrete memoryless channel are selected according to the probability distribution $\{p(x_j), j=0,1,\ldots,J-1\}$. In other words, the even that the channel input $X=x_j$ occurs with probability x

$$p(x_j) = P(X = x_j) \qquad for\ j = 0,1, \ldots, J-1$$

Having specified the random variable X denoting the channel input, we may now specify the second random variable Y denoting the channel output. The joint probability density of the random variables X and Y is given by

$$p(x_j, y_k) = P(X = x_j, Y = y_k) = p(y_k|x_j)p(x_j)$$

The marginal probability density of the output random variable Y is obtained by averaging out the dependence of $p(x_j, y_k)$ on x_j, as shown by

$$p(y_k) = \sum_{j=0}^{J-1} p(y_k|x_j)p(x_j) \qquad for\ k = 0,1, \dots, K-1$$

Please note that the following pages have been directly picked from Communication Systems - Simon Haykins, 4th Edition, Chapter 9. You will find the required pre-requisites in the content before this. The following is all relevant to your course and it is important that you study all of it.

Binary Symmetric Channel

The *binary symmetric channel* is of great theoretical interest and practical importance. It is a special case of the discrete memoryless channel with $J = K = 2$. The channel has two input symbols ($x_0 = 0, x_1 = 1$) and two output symbols ($y_0 = 0, y_1 = 1$). The channel is symmetric because the probability of receiving a 1 if a 0 is sent is the same as the probability of receiving a 0 if a 1 is sent. This conditional probability of error is denoted by p. The *transition probability diagram* of a binary symmetric channel is as shown in Figure 9.8. ◀

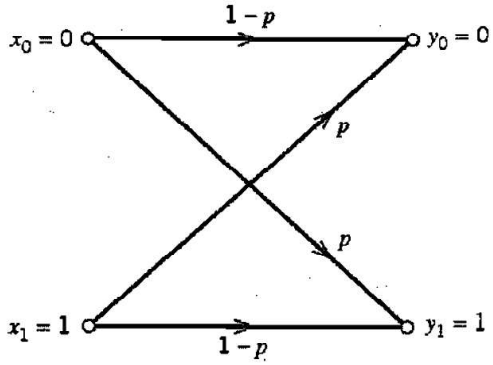


FIGURE 9.8 Transition probability diagram of binary symmetric channel.

The transition matrix for a binary symmetric channel is hence given by

$$P = \begin{bmatrix} 1-p & p \\ p & 1-p \end{bmatrix}$$

Channel Capacity

Consider a discrete memoryless channel with input alphabet $\mathcal{X}$, output alphabet $\mathcal{Y}$, and transition probabilities $p(y_k|x_j)$, where $j = 0, 1, \ldots, J-1$ and $k = 0, 1, \ldots, K-1$. The mutual information of the channel is defined by the first line of Equation (9.49), which is reproduced here for convenience:

$$I(\mathcal{X}; \mathcal{Y}) = \sum_{k=0}^{K-1} \sum_{j=0}^{J-1} p(x_j, y_k) \log_2\left[\frac{p(y_k|x_j)}{p(y_k)}\right]$$

Here we note that [see Equation (9.38)]

$$p(x_j, y_k) = p(y_k|x_j)p(x_j)$$

Also, from Equation (9.39), we have

$$p(y_k) = \sum_{j=0}^{J-1} p(y_k|x_j)p(x_j)$$

From these three equations we see that it is necessary for us to know the input probability distribution $\{p(x_j) | j = 0, 1, \ldots, J-1\}$ so that we may calculate the mutual information $I(\mathcal{X}; \mathcal{Y})$. The mutual information of a channel therefore depends not only on the channel but also on the way in which the channel is used.

The input probability distribution $\{p(x_j)\}$ is obviously independent of the channel. We can then maximize the mutual information $I(\mathcal{X}; \mathcal{Y})$ of the channel with respect to $\{p(x_j)\}$. Hence, *we define the channel capacity of a discrete memoryless channel as the maximum mutual information* $I(\mathcal{X}; \mathcal{Y})$ *in any single use of the channel (i.e., signaling interval), where the maximization is over all possible input probability distributions* $\{p(x_j)\}$ *on* $\mathcal{X}$. The channel capacity is commonly denoted by C. We thus write

$$C = \max_{\{p(x_j)\}} I(\mathcal{X}; \mathcal{Y}) \tag{9.59}$$

The channel capacity C is measured in *bits per channel use*, or *bits per transmission*.

Note that the channel capacity C is a function only of the transition probabilities $p(y_k|x_j)$, which define the channel. The calculation of C involves maximization of the mutual information $I(\mathcal{X}; \mathcal{Y})$ over J variables [i.e., the input probabilities $p(x_0), \ldots, p(x_{J-1})$] subject to two constraints:

$$p(x_j) \geq 0 \text{ for all } j$$

and

$$\sum_{j=0}^{J-1} p(x_j) = 1$$

In general, the variational problem of finding the channel capacity C is a challenging task.

▶ EXAMPLE 9.5 Binary Symmetric Channel (Revisited)

Consider again the *binary symmetric channel*, which is described by the *transition probability diagram* of Figure 9.8. This diagram is uniquely defined by the conditional probability of error p.

The entropy $H(X)$ is maximized when the channel input probability $p(x_0) = p(x_1) = 1/2$, where x_0 and x_1 are each 0 or 1. The mutual information $I(\mathcal{X}; \mathcal{Y})$ is similarly maximized, so that we may write

$$C = I(\mathcal{X}; \mathcal{Y})\big|_{p(x_0)=p(x_1)=1/2}$$

From Figure 9.8, we have

$$p(y_0|x_1) = p(y_1|x_0) = p$$

and

$$p(y_0|x_0) = p(y_1|x_1) = 1 - p$$

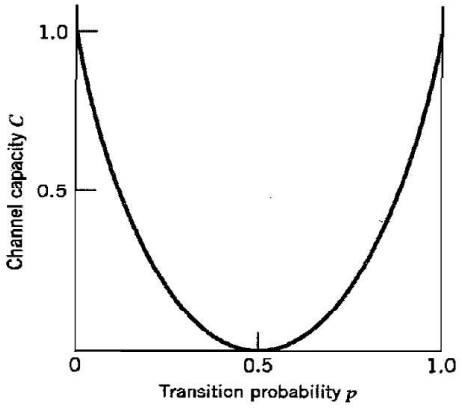


FIGURE 9.10 Variation of channel capacity of a binary symmetric channel with transition probability p.

Therefore, substituting these channel transition probabilities into Equation (9.49) with $J = K = 2$, and then setting the input probability $p(x_0) = p(x_1)$ in accordance with Equation (9.59), we find that the capacity of the binary symmetric channel is

$$C = 1 + p \log_2 p + (1 - p) \log_2(1 - p) \tag{9.60}$$

Using the definition of the entropy function given in Equation (9.16), we may reduce Equation (9.60) to

$$C = 1 - H(p)$$

The channel capacity C varies with the probability of error (transition probability) p in a convex manner as shown in Figure 9.10, which is symmetric about $p = 1/2$. Comparing the curve in this figure with that in Figure 9.2, we may make the following observations:

1. When the channel is *noise free*, permitting us to set $p = 0$, the channel capacity C attains its maximum value of one bit per channel use, which is exactly the information in each channel input. At this value of p, the entropy function $H(p)$ attains its minimum value of zero.
2. When the conditional probability of error $p = 1/2$ due to noise, the channel capacity C attains its minimum value of zero, whereas the entropy function $H(p)$ attains its maximum value of unity; in such a case the channel is said to be *useless*. ◀

Channel Coding Theorem

The inevitable presence of *noise* in a channel causes discrepancies (errors) between the output and input data sequences of a digital communication system. For a relatively noisy channel (e.g., wireless communication channel), the probability of error may reach a value as high as 10^{-1}, which means that (on the average) only 9 out of 10 transmitted bits are received correctly. For many applications, this *level of reliability* is unacceptable. Indeed, a probability of error equal to 10^{-6} or even lower is often a necessary requirement. To achieve such a high level of performance, we resort to the use of channel coding.

The design goal of channel coding is to increase the resistance of a digital communication system to channel noise. Specifically, *channel coding* consists of *mapping* the incoming data sequence into a channel input sequence, and *inverse mapping* the channel output sequence into an output data sequence in such a way that the overall effect of channel noise on the system is minimized. The first mapping operation is performed in the transmitter by a *channel encoder*, whereas the inverse mapping operation is performed in the receiver by a *channel decoder*, as shown in the block diagram of Figure 9.11; to simplify the exposition, we have not included source encoding (before channel encoding) and source decoding (after channel decoding) in Figure 9.11.

The channel encoder and channel decoder in Figure 9.11 are both under the designer's control and should be designed to optimize the overall reliability of the communication system. The approach taken is to introduce *redundancy* in the channel encoder so as to reconstruct the original source sequence as accurately as possible. Thus, in a rather loose sense, we may view channel coding as the *dual* of source coding in that the former introduces controlled redundancy to improve reliability, whereas the latter reduces redundancy to improve efficiency.

The subject of channel coding is treated in detail in Chapter 10. For the purpose of our present discussion, it suffices to confine our attention to *block codes*. In this class of codes, the message sequence is subdivided into sequential blocks each k bits long, and each k-bit block is *mapped* into an n-bit block, where $n > k$. The number of redundant bits added by the encoder to each transmitted block is $n - k$ bits. The ratio k/n is called the *code rate*. Using r to denote the code rate, we may thus write

$$r = \frac{k}{n}$$

where, of course, r is less than unity. For a prescribed k, the code rate r (and therefore the system's coding efficiency) approaches zero as the block length n approaches infinity.

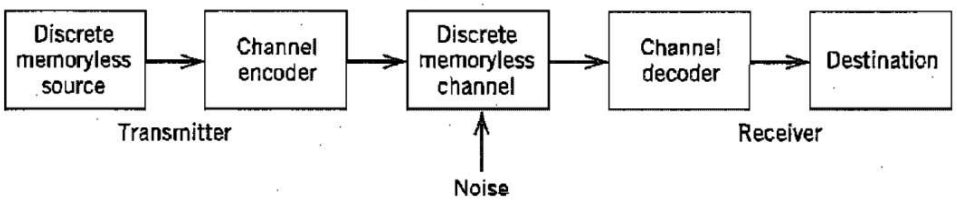


FIGURE 9.11 Block diagram of digital communication system.

The accurate reconstruction of the original source sequence at the destination requires that the *average probability of symbol error* be arbitrarily low. This raises the following important question: Does there exist a channel coding scheme such that the probability that a message bit will be in error is less than any positive number ϵ (i.e., as small as we want it), and yet the channel coding scheme is efficient in that the code rate need not be too small? The answer to this fundamental question is an emphatic "yes." Indeed, the answer to the question is provided by Shannon's second theorem in terms of the channel capacity C, as described in what follows. Up until this point, *time* has not played an important role in our discussion of channel capacity. Suppose then the discrete memoryless source in Figure 9.11 has the source alphabet $\mathcal{S}$ and entropy $H(\mathcal{S})$ bits per source symbol. We assume that the source emits symbols once every T_s seconds. Hence, the *average information rate* of the source is $H(\mathcal{S})/T_s$ bits per second. The decoder delivers decoded symbols to the destination from the source alphabet $\mathcal{S}$ and at the same source rate of one symbol every T_s seconds. The discrete memoryless channel has a channel capacity equal to C bits per use of the channel. We assume that the channel is capable of being used once every T_c seconds. Hence, the *channel capacity per unit time is* C/T_c bits per second, which represents the maximum rate of information transfer over the channel. We are now ready to state Shannon's second theorem, known as the channel coding theorem.

Specifically, the *channel coding theorem*[8] for a discrete memoryless channel is stated in two parts as follows.

(i) Let a discrete memoryless source with an alphabet $\mathcal{S}$ have entropy $H(\mathcal{S})$ and produce symbols once every T_s seconds. Let a discrete memoryless channel have capacity C and be used once every T_c seconds. Then, if

$$\frac{H(\mathcal{S})}{T_s} \leq \frac{C}{T_c} \tag{9.61}$$

there exists a coding scheme for which the source output can be transmitted over the channel and be reconstructed with an arbitrarily small probability of error. The parameter C/T_c is called the *critical rate*. When Equation (9.61) is satisfied with the equality sign, the system is said to be signaling at the critical rate.

(ii) Conversely, if

$$\frac{H(\mathcal{S})}{T_s} > \frac{C}{T_c}$$

it is not possible to transmit information over the channel and reconstruct it with an arbitrarily small probability of error.

The channel coding theorem is the single most important result of information theory. The theorem specifies the channel capacity C as a *fundamental limit* on the rate at which the transmission of reliable error-free messages can take place over a discrete memoryless channel. However, it is important to note the following:

- The channel coding theorem does not show us how to construct a good code. Rather, the theorem should be viewed as an *existence proof* in the sense that it tells us that if the condition of Equation (9.61) is satisfied, then good codes do exist. (Later in Chapter 10 we describe several good codes for discrete memoryless channels.)
- The theorem does not have a precise result for the probability of symbol error after decoding the channel output. Rather, it tells us that the probability of symbol error tends to zero as the length of the code increases, again provided that the condition of Equation (9.61) is satisfied.

Application of the Channel Coding Theorem to Binary Symmetric Channels

Consider a discrete memoryless source that emits equally likely binary symbols (0s and 1s) once every T_s seconds. With the source entropy equal to one bit per source symbol (see Example 9.1), the information rate of the source is $(1/T_s)$ bits per second. The source sequence is applied to a channel encoder with code rate r. The channel encoder produces a symbol once every T_c seconds. Hence, the encoded symbol transmission rate is $(1/T_c)$ symbols per second. The channel encoder engages a binary symmetric channel once every T_c seconds. Hence, the channel capacity per unit time is (C/T_c) bits per second, where C is determined by the prescribed channel transition probability p in accordance with Equation (9.60). Accordingly, the channel coding theorem [part (i)] implies that if

$$\frac{1}{T_s} \leq \frac{C}{T_c} \tag{9.62}$$

the probability of error can be made arbitrarily low by the use of a suitable channel encoding scheme. But the ratio T_c/T_s equals the code rate of the channel encoder:

$$r = \frac{T_c}{T_s} \tag{9.63}$$

Hence, we may restate the condition of Equation (9.62) simply as

$$r \leq C \tag{9.64}$$

That is, for $r \leq C$, there exists a code (with code rate less than or equal to C) capable of achieving an arbitrarily low probability of error.

▶ EXAMPLE 9.6 Repetition Code

In this example, we present a graphical interpretation of the channel coding theorem. We also bring out a surprising aspect of the theorem by taking a look at a simple coding scheme.

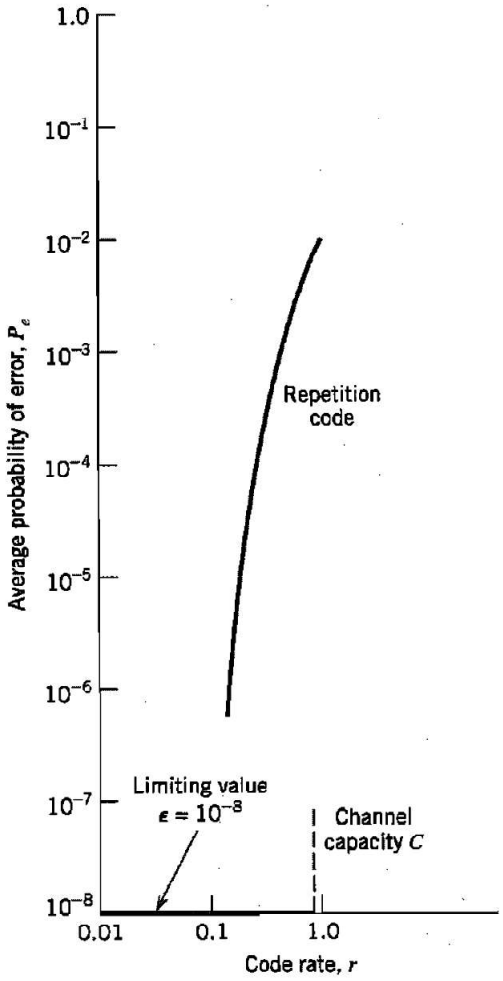


FIGURE 9.12 Illustrating significance of the channel coding theorem.

TABLE 9.3 *Average probability of error for repetition code*

Code Rate, r = 1/n	*Average Probability of Error*, P_e
1	10^{-2}
$\frac{1}{3}$	3×10^{-4}
$\frac{1}{5}$	10^{-6}
$\frac{1}{7}$	4×10^{-7}
$\frac{1}{9}$	10^{-8}
$\frac{1}{11}$	5×10^{-10}

Consider first a binary symmetric channel with transition probability $p = 10^{-2}$. For this value of p, we find from Equation (9.60) that the channel capacity $C = 0.9192$. Hence, from the channel coding theorem, we may state that for any $\epsilon > 0$ and $r \leq 0.9192$, there exists a code of large enouigh length n and code rate r, and an appropriate decoding algorithm, such that when the coded bit stream is sent over the given channel, the average probability of channel decoding error is less than ϵ. This result is depicted in Figure 9.12 for the limiting value $\epsilon = 10^{-8}$.

To put the significance of this result in perspective, consider next a simple coding scheme that involves the use of a *repetition code*, in which each bit of the message is repeated several times. Let each bit (0 or 1) be repeated n times, where $n = 2m + 1$ is an odd integer. For example, for $n = 3$, we transmit 0 and 1 as 000 and 111, respectively. Intuitively, it would seem logical to use a *majority rule* for decoding, which operates as follows: *If in a block of n received bits (representing one bit of the message), the number of 0s exceeds the number of 1s, the decoder decides in favor of a 0. Otherwise, it decides in favor of a 1.* Hence, an error occurs when $m + 1$ or more bits out of $n = 2m + 1$ bits are received incorrectly. Because of the assumed symmetric nature of the channel, the *average probability of error* P_e is independent of the *a priori* probabilities of 0 and 1. Accordingly, we find that P_e is given by (see Problem 9.24)

$$P_e = \sum_{i=m+1}^{n} \binom{n}{i} p^i (1 - p)^{n-i} \tag{9.65}$$

where p is the transition probability of the channel.

Table 9.3 gives the average probability of error P_e for a repetition code, which is calculated by using Equation (9.65) for different values of the code rate r. The values given here assume the use of a binary symmetric channel with transition probability $p = 10^{-2}$. The improvement in reliability displayed in Table 9.3 is achieved at the cost of decreasing code rate. The results of this table are also shown plotted as the curve labeled "repetition code" in Figure 9.12. This curve illustrates the *exchange of code rate for message reliability*, which is a characteristic of repetition codes.

This example highlights the unexpected result presented to us by the channel coding theorem. The result is that it is not necessary to have the code rate r approach zero (as in the case of repetition codes) so as to achieve more and more reliable operation of the communication link. The theorem merely requires that the code rate be less than the channel capacity C. ◀

Differential Entropy

The sources and channels considered in our discussion of information-theoretic concepts thus far have involved ensembles of random variables that are *discrete* in amplitude. In this section, we extend some of these concepts to *continuous* random variables and random vectors. The motivation for doing so is to pave the way for the description of another fundamental limit in information theory, which we take up in Section 9.10.

Consider a continuous random variable X with the *probability density function* $f_X(x)$. By analogy with the entropy of a discrete random variable, we introduce the following definition:

$$h(X) = \int_{-\infty}^{\infty} f_X(x) \log_2\left[\frac{1}{f_X(x)}\right] dx \tag{9.66}$$

We refer to $h(X)$ as the *differential entropy of* X to distinguish it from the ordinary or *absolute entropy*. We do so in recognition of the fact that although $h(X)$ is a useful mathematical quantity to know, it is *not* in any sense a measure of the randomness of X. Nevertheless, we justify the use of Equation (9.66) in what follows. We begin by viewing the continuous random variable X as the limiting form of a discrete random variable that assumes the value $x_k = k\,\Delta x$, where $k = 0, \pm 1, \pm 2, \ldots$, and Δx approaches zero. By definition, the continuous random variable X assumes a value in the interval $[x_k, x_k + \Delta x]$ with probability $f_X(x_k)\,\Delta x$. Hence, permitting Δx to approach zero, the ordinary entropy of the continuous random variable X may be written in the limit as follows:

$$\begin{aligned} H(X) &= \lim_{\Delta x \to 0} \sum_{k=-\infty}^{\infty} f_X(x_k)\,\Delta x \log_2\left(\frac{1}{f_X(x_k)\,\Delta x}\right) \\ &= \lim_{\Delta x \to 0} \left[\sum_{k=-\infty}^{\infty} f_X(x_k) \log_2\left(\frac{1}{f_X(x_k)}\right)\Delta x - \log_2 \Delta x \sum_{k=-\infty}^{\infty} f_X(x_k)\,\Delta x\right] \\ &= \int_{-\infty}^{\infty} f_X(x) \log_2\left(\frac{1}{f_X(x)}\right) dx - \lim_{\Delta x \to 0} \log_2 \Delta x \int_{-\infty}^{\infty} f_X(x)\,dx \\ &= h(X) - \lim_{\Delta x \to 0} \log_2 \Delta x \end{aligned} \tag{9.67}$$

where, in the last line, we have made use of Equation (9.66) and the fact that the total area under the curve of the probability density function $f_X(x)$ is unity. In the limit as Δx approaches zero, $-\log_2 \Delta x$ approaches infinity. This means that the entropy of a continuous random variable is infinitely large. Intuitively, we would expect this to be true, because a continuous random variable may assume a value anywhere in the interval $(-\infty, \infty)$ and the uncertainty associated with the variable is on the order of infinity. We avoid the problem associated with the term $\log_2 \Delta x$ by adopting $h(X)$ as a differential entropy, with the term $-\log_2 \Delta x$ serving as reference. Moreover, since the information transmitted over a channel is actually the difference between two entropy terms that have a common reference, the information will be the same as the difference between the corresponding differential entropy terms. We are therefore perfectly justified in using the term $h(X)$, defined in Equation (9.66), as the differential entropy of the continuous random variable X.

When we have a continuous random vector $\mathbf{X}$ consisting of n random variables $X_1, X_2, \ldots, X_n$, we define the differential entropy of $\mathbf{X}$ as the *n-fold integral*

$$h(\mathbf{X}) = \int_{-\infty}^{\infty} f_{\mathbf{X}}(\mathbf{x}) \log_2\left[\frac{1}{f_{\mathbf{X}}(\mathbf{x})}\right] d\mathbf{x} \tag{9.68}$$

where $f_{\mathbf{X}}(\mathbf{x})$ is the *joint probability density function* of $\mathbf{X}$.

▶ EXAMPLE 9.7 Uniform Distribution

Consider a random variable X uniformly distributed over the interval $(0, a)$. The probability density function of X is

$$f_X(x) = \begin{cases} \frac{1}{a}, & 0 < x < a \\ 0, & \text{otherwise} \end{cases}$$

Applying Equation (9.66) to this distribution, we get

$$\begin{aligned} h(X) &= \int_0^a \frac{1}{a} \log(a)\, dx \\ &= \log a \end{aligned} \tag{9.69}$$

Note that $\log a < 0$ for $a < 1$. Thus this example shows that, unlike a discrete random variable, the differential entropy of a continuous random variable can be negative. ◀

Gaussian Distribution

Consider an arbitrary pair of random variables X and Y, whose probability density functions are respectively denoted by $f_Y(x)$ and $f_X(x)$ where x is merely a dummy variable. Adapting the fundamental inequality of Equation (9.12) to the situation at hand, we may write[9]

$$\int_{-\infty}^{\infty} f_Y(x) \log_2\left(\frac{f_X(x)}{f_Y(x)}\right) dx \leq 0 \tag{9.70}$$

or, equivalently,

$$-\int_{-\infty}^{\infty} f_Y(x) \log_2 f_Y(x)\, dx \leq -\int_{-\infty}^{\infty} f_Y(x) \log_2 f_X(x)\, dx \tag{9.71}$$

The quantity on the left-hand side of Equation (9.71) is the differential entropy of the random variable Y; hence,

$$h(Y) \leq -\int_{-\infty}^{\infty} f_Y(x) \log_2 f_X(x)\, dx \tag{9.72}$$

Suppose now the random variables X and Y are described as follows:

- The random variables X and Y have the *same mean* μ and the *same variance* σ^2.
- The random variable X is *Gaussian distributed* as shown by

$$f_X(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{9.73}$$

Hence, substituting Equation (9.73) into Equation (9.72), and changing the base of the logarithm from 2 to $e = 2.7183$, we get

$$h(Y) \leq -\log_2 e \int_{-\infty}^{\infty} f_Y(x)\left(-\frac{(x-\mu)^2}{2\sigma^2} - \log(\sqrt{2\pi}\sigma)\right) dx \tag{9.74}$$

We now recognize the following properties of the random variable Y (given that its mean is μ and its variance is σ^2):

$$\int_{-\infty}^{\infty} f_Y(x)\, dx = 1$$

$$\int_{-\infty}^{\infty} (x-\mu)^2 f_Y(x)\, dx = \sigma^2$$

We may therefore simplify Equation (9.74) as

$$h(Y) \leq \tfrac{1}{2} \log_2(2\pi e\sigma^2) \tag{9.75}$$

The quantity on the right-hand side of Equation (9.75) is in fact the differential entropy of the Gaussian random variable X:

$$h(X) = \tfrac{1}{2} \log_2(2\pi e\sigma^2) \tag{9.76}$$

Finally, combining Equations (9.75) and (9.76), we may write

$$h(Y) \leq h(X), \qquad \begin{cases} X\text{: Gaussian random variable} \\ Y\text{: another random variable} \end{cases} \tag{9.77}$$

where equality holds if, and only if, $Y = X$.

We may now summarize the results of this important example as two entropic properties of a Gaussian random variable:

1. For a finite variance σ^2, the Gaussian random variable has the largest differential entropy attainable by any random variable.
2. The entropy of a Gaussian random variable X is uniquely determined by the variance of X (i.e., it is independent of the mean of X).

Indeed, it is because of Property 1 that the Gaussian channel model is so widely used as a conservative model in the study of digital communication systems. ◀

■ Mutual Information

Consider next a pair of continuous random variables X and Y. By analogy with Equation (9.47), we define the *mutual information* between the random variables X and Y as follows:

$$I(X; Y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f_{X,Y}(x,y) \log_2 \left[\frac{f_X(x|y)}{f_X(x)} \right] dx\, dy \tag{9.78}$$

where $f_{X,Y}(x, y)$ is the joint probability density function of X and Y, and $f_X(x|y)$ is the conditional probability density function of X, given that $Y = y$. Also, by analogy with Equations (9.45), (9.50), (9.43), and (9.44) we find that the mutual information $I(X; Y)$ has the following properties:

$$1.\ I(X; Y) = I(Y; X) \tag{9.79}$$

$$2.\ I(X; Y) \geq 0 \tag{9.80}$$

$$3.\ I(X; Y) = h(X) - h(X|Y) = h(Y) - h(Y|X) \tag{9.81}$$

The parameter $h(X)$ is the differential entropy of X; likewise for $h(Y)$. The parameter $h(X|Y)$ is the *conditional differential entropy* of X, given Y; it is defined by the double integral (see Equation (9.41))

$$h(X|Y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f_{X,Y}(x, y) \log_2 \left[\frac{1}{f_X(x|y)} \right] dx\, dy \tag{9.82}$$

The parameter $h(Y|X)$ is the conditional differential entropy of Y, given X; it is defined in a manner similar to $h(X|Y)$.

Information Capacity Theorem

In this section, we use the idea of mutual information to formulate the information capacity theorem for *band-limited, power-limited Gaussian channels.* To be specific, consider a zero-mean stationary process $X(t)$ that is band-limited to B hertz. Let X_k, $k = 1, 2, \ldots, K$, denote the continuous random variables obtained by uniform sampling of the process $X(t)$ at the Nyquist rate of $2B$ samples per second. These samples are transmitted in T seconds over a noisy channel, also band-limited to B hertz. Hence, the number of samples, K, is given by

$$K = 2BT \tag{9.83}$$

We refer to X_k as a sample of the *transmitted signal.* The channel output is perturbed by *additive white Gaussian noise* (AWGN) of zero mean and power spectral density $N_0/2$. The noise is band-limited to B hertz. Let the continuous random variables Y_k, $k = 1, 2, \ldots, K$ denote samples of the received signal, as shown by

$$Y_k = X_k + N_k, \qquad k = 1, 2, \ldots, K \tag{9.84}$$

The noise sample N_k is Gaussian with zero mean and variance given by

$$\sigma^2 = N_0 B \tag{9.85}$$

We assume that the samples Y_k, $k = 1, 2, \ldots, K$ are statistically independent.

A channel for which the noise and the received signal are as described in Equations (9.84) and (9.85) is called a *discrete-time, memoryless Gaussian channel.* It is modeled as in Figure 9.13. To make meaningful statements about the channel, however, we have to assign a *cost* to each channel input. Typically, the transmitter is *power limited*; it is therefore reasonable to define the cost as

$$E[X_k^2] = P, \qquad k = 1, 2, \ldots, K \tag{9.86}$$

where P is the *average transmitted power.* The *power-limited Gaussian channel* described herein is of not only theoretical but also practical importance in that it models many communication channels, including line-of-sight radio and satellite links.

The *information capacity* of the channel is defined as the maximum of the mutual information between the channel input X_k and the channel output Y_k over all distributions on the input X_k that satisfy the power constraint of Equation (9.86). Let $I(X_k; Y_k)$ denote

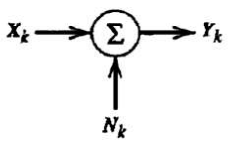


FIGURE 9.13 Model of discrete-time, memoryless Gaussian channel.

the mutual information between X_k and Y_k. We may then define the information capacity of the channel as

$$C = \max_{f_{X_k}(x)} \{I(X_k; Y_k) : E[X_k^2] = P\} \tag{9.87}$$

where the maximization is performed with respect to $f_{X_k}(x)$, the probability density function of X_k.

The mutual information $I(X_k; Y_k)$ can be expressed in one of the two equivalent forms shown in Equation (9.81). For the purpose at hand, we use the second line of this equation and so write

$$I(X_k; Y_k) = h(Y_k) - h(Y_k | X_k) \tag{9.88}$$

Since X_k and N_k are independent random variables, and their sum equals Y_k, as in Equation (9.84), we find that the conditional differential entropy of Y_k, given X_k, is equal to the differential entropy of N_k (see Problem 9.28):

$$h(Y_k \mid X_k) = h(N_k) \tag{9.89}$$

Hence, we may rewrite Equation (9.88) as

$$I(X_k; Y_k) = h(Y_k) - h(N_k) \tag{9.90}$$

Since $h(N_k)$ is independent of the distribution of X_k, maximizing $I(X_k; Y_k)$ in accordance with Equation (9.87) requires maximizing $h(Y_k)$, the differential entropy of sample Y_k of the received signal. For $h(Y_k)$ to be maximum, Y_k has to be a Gaussian random variable (see Example 9.8). That is, the samples of the received signal represent a noiselike process. Next, we observe that since N_k is Gaussian by assumption, the sample X_k of the transmitted signal must be Gaussian too. We may therefore state that the maximization specified in Equation (9.87) is attained by choosing the samples of the transmitted signal from a noiselike process of average power P. Correspondingly, we may reformulate Equation (9.87) as

$$C = I(X_k; Y_k) : X_k \text{ Gaussian}, \qquad E[X_k^2] = P \tag{9.91}$$

where the mutual information $I(X_k; Y_k)$ is defined in accordance with Equation (9.90).

For the evaluation of the information capacity C, we proceed in three stages:

1. The variance of sample Y_k of the received signal equals $P + \sigma^2$. Hence, the use of Equation (9.76) yields the differential entropy of Y_k as

$$h(Y_k) = \tfrac{1}{2} \log_2[2\pi e(P + \sigma^2)] \tag{9.92}$$

2. The variance of the noise sample N_k equals σ^2. Hence, the use of Equation (9.76) yields the differential entropy of N_k as

$$h(N_k) = \tfrac{1}{2} \log_2(2\pi e\sigma^2) \tag{9.93}$$

3. Substituting Equations (9.92) and (9.93) into Equation (9.90) and recognizing the definition of information capacity given in Equation (9.91), we get the desired result:

$$C = \frac{1}{2} \log_2\left(1 + \frac{P}{\sigma^2}\right) \text{ bits per transmission} \tag{9.94}$$

With the channel used K times for the transmission of K samples of the process $X(t)$ in T seconds, we find that the information capacity per unit time is (K/T) times the result

given in Equation (9.94). The number K equals $2BT$, as in Equation (9.83). Accordingly, we may express the information capacity in the equivalent form:

$$C = B \log_2\left(1 + \frac{P}{N_0 B}\right) \text{ bits per second} \tag{9.95}$$

where we have used Equation (9.85) for the noise variance σ^2.

Based on the formula of Equation (9.95), we may now state Shannon's third (and most famous) theorem, the *information capacity theorem*,[10] as follows:

The information capacity of a continuous channel of bandwidth B hertz, perturbed by additive white Gaussian noise of power spectral density $N_0/2$ and limited in bandwidth to B, is given by

$$C = B \log_2\left(1 + \frac{P}{N_0 B}\right) \text{ bits per second}$$

where P is the average transmitted power.

The information capacity theorem is one of the most remarkable results of information theory for, in a single formula, it highlights most vividly the interplay among three key system parameters: channel bandwidth, average transmitted power (or, equivalently, average received signal power), and noise power spectral density at the channel output. The dependence of information capacity C on channel bandwidth B is *linear*, whereas its dependence on signal-to-noise ratio P/N_0B is *logarithmic*. Accordingly, *it is easier to increase the information capacity of a communication channel by expanding its bandwidth than increasing the transmitted power for a prescribed noise variance.*

The theorem implies that, for given average transmitted power P and channel bandwidth B, we can transmit information at the rate of C bits per second, as defined in Equation (9.95), with arbitrarily small probability of error by employing sufficiently complex encoding systems. It is not possible to transmit at a rate higher than C bits per second by any encoding system without a definite probability of error. Hence, the channel capacity theorem defines the *fundamental limit* on the rate of error-free transmission for a power-limited, band-limited Gaussian channel. To approach this limit, however, the transmitted signal must have statistical properties approximating those of white Gaussian noise.

CHAPTER 10

ERROR-CONTROL CODING

This chapter is the natural sequel to the preceding chapter on Shannon's information theory. In particular, in this chapter we present error-control coding techniques that provide different ways of implementing Shannon's channel-coding theorem. Each error-control coding technique involves the use of a channel encoder in the transmitter and a decoding algorithm in the receiver.

The error-control coding techniques described herein include the following important classes of codes:

- *Linear block codes.*
- *Cyclic codes.*
- *Convolutional codes.*
- *Compound codes exemplified by turbo codes and low-density parity-check codes, and their irregular variants.*

10.1 *Introduction*

The task facing the designer of a digital communication system is that of providing a cost-effective facility for transmitting information from one end of the system at a rate and a level of reliability and quality that are acceptable to a user at the other end. The two key system parameters available to the designer are transmitted signal power and channel bandwidth. These two parameters, together with the power spectral density of receiver noise, determine the signal energy per bit-to-noise power spectral density ratio E_b/N_0. In Chapter 6, we showed that this ratio uniquely determines the bit error rate for a particular modulation scheme. Practical considerations usually place a limit on the value that we can assign to E_b/N_0. Accordingly, in practice, we often arrive at a modulation scheme and find that it is not possible to provide acceptable data quality (i.e., low enough error performance). For a fixed E_b/N_0, the only practical option available for changing data quality from problematic to acceptable is to use *error-control coding*.

Another practical motivation for the use of coding is to reduce the required E_b/N_0 for a fixed bit error rate. This reduction in E_b/N_0 may, in turn, be exploited to reduce the required transmitted power or reduce the hardware costs by requiring a smaller antenna size in the case of radio communications.

Error control[1] for data integrity may be exercised by means of *forward error correction* (FEC). Figure 10.1*a* shows the model of a digital communication system using such an approach. The discrete source generates information in the form of binary symbols. The *channel encoder* in the transmitter accepts message bits and adds *redundancy* according to a prescribed rule, thereby producing encoded data at a higher bit rate. The *channel*

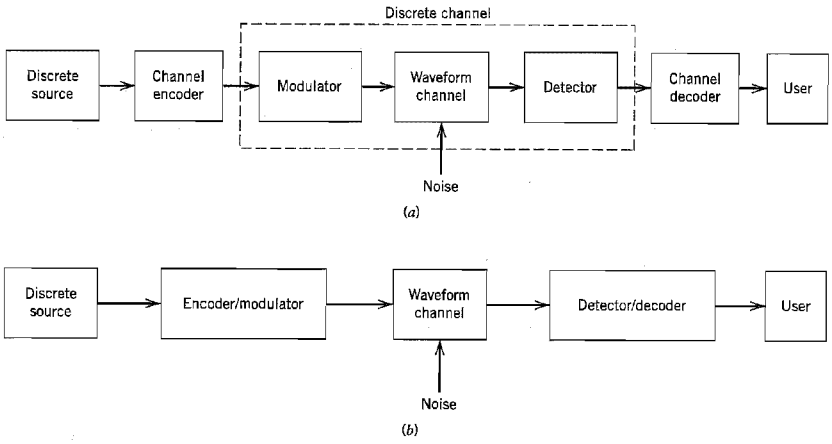


FIGURE 10.1 Simplified models of digital communication system. (*a*) Coding and modulation performed separately. (*b*) Coding and modulation combined.

decoder in the receiver exploits the redundancy to decide which message bits were actually transmitted. The combined goal of the channel encoder and decoder is to minimize the effect of channel noise. That is, the number of errors between the channel encoder input (derived from the source) and the channel decoder output (delivered to the user) is minimized.

For a fixed modulation scheme, the addition of redundancy in the coded messages implies the need for increased transmission bandwidth. Moreover, the use of error-control coding adds *complexity* to the system, especially for the implementation of decoding operations in the receiver. Thus, the design trade-offs in the use of error-control coding to achieve acceptable error performance include considerations of bandwidth and system complexity.

There are many different error-correcting codes (with roots in diverse mathematical disciplines) that we can use. Historically, these codes have been classified into *block codes* and *convolutional codes*. The distinguishing feature for this particular classification is the presence or absence of *memory* in the encoders for the two codes.

To generate an (n, k) block code, the channel encoder accepts information in successive k-bit *blocks*; for each block, it adds $n - k$ redundant bits that are algebraically related to the k message bits, thereby producing an overall encoded block of n bits, where $n > k$. The n-bit block is called a *code word*, and n is called the *block length* of the code. The channel encoder produces bits at the rate $R_0 = (n/k)R_s$, where R_s is the bit rate of the information source. The dimensionless ratio $r = k/n$ is called the *code rate*, where $0 < r < 1$. The bit rate R_0, coming out of the encoder, is called the *channel data rate*. Thus, the code rate is a dimensionless ratio, whereas the data rate produced by the source and the channel data rate are both measured in bits per second.

In a convolutional code, the encoding operation may be viewed as the *discrete-time convolution* of the input sequence with the impulse response of the encoder. The duration of the impulse response equals the memory of the encoder. Accordingly, the encoder for a convolutional code operates on the incoming message sequence, using

a "sliding window" equal in duration to its own memory. This, in turn, means that in a convolutional code, unlike a block code, the channel encoder accepts message bits as a continuous sequence and thereby generates a continuous sequence of encoded bits at a higher rate.

In the model depicted in Figure 10.1*a*, the operations of channel coding and modulation are performed separately in the transmitter; likewise for the operations of detection and decoding in the receiver. When, however, bandwidth efficiency is of major concern, the most effective method of implementing forward error-control correction coding is to combine it with modulation as a single function, as shown in Figure 10.1*b*. In such an approach, coding is redefined as a process of imposing certain patterns on the transmitted signal.

■ AUTOMATIC-REPEAT REQUEST

Feed-forward error correction (FEC) relies on the controlled use of redundancy in the transmitted code word for both the *detection and correction* of errors incurred during the course of transmission over a noisy channel. Irrespective of whether the decoding of the received code word is successful, no further processing is performed at the receiver. Accordingly, channel coding techniques suitable for FEC require only a *one-way link* between the transmitter and receiver.

There is another approach known as *automatic-repeat request (ARQ)*[2] for solving the error-control problem. The underlying philosophy of ARQ is quite different from that of FEC. Specifically, ARQ uses redundancy merely for the purpose of *error detection*. Upon the detection of an error in a transmitted code word, the receiver requests a repeat transmission of the corrupted code word, which necessitates the use of a *return path* (i.e., a feedback channel). As such, ARQ can be used only on *half-duplex* or *full-duplex links*. In a half-duplex link, data transmission over the link can be made in either direction but *not* simultaneously. On the other hand, in a full-duplex link, it is possible for data transmission to proceed over the link in both directions simultaneously.

A half-duplex link uses the simplest ARQ scheme known as the *stop-and-wait strategy*. In this approach, a block of message bits is encoded into a code word and transmitted over the channel. The transmitter then stops and waits for feedback from the receiver. The feedback signal can be acknowledgment of a correct receipt of the code word or a request for transmission of the code word because of an error in its decoding. In the latter case, the transmitter resends the code word in question before moving onto the next block of message bits.

The idling problem in stop-and-wait ARQ results in reduced data throughput, which is alleviated in another type of ARQ known as *continuous ARQ with pullback*. This second strategy uses a full-duplex link, thereby permitting the receiver to send a feedback signal while the transmitter is engaged in sending code words over the forward channel. Specifically, the transmitter continues to send a succession of code words until it receives a request from the receiver (on the feedback channel) for a retransmission. At that point, the transmitter stops, pulls back to the particular code word that was not decoded correctly by the receiver, and retransmits the complete sequence of code words starting with the corrupted one.

In a refined version of continuous ARQ known as the *continuous ARQ with selective repeat*, data throughout is improved further by only retransmitting the code word that was received with detected errors. In other words, the need for retransmitting the successfully received code words following the corrupted code word is eliminated.

The three types of ARQ described here offer trade-offs of their own between the need for a half-duplex or full-duplex link and the requirement for efficient use of communication resources. In any event, they all rely on two premises:

- Error detection, which makes the design of the decoder relatively simple.
- Noiseless feedback channel, which is not a severe restriction because the rate of information flow over the feedback channel is typically quite low.

For these reasons, ARQ is widely used in computer-communication systems.

Nevertheless, the fact that FEC requires only one-way links for its operation makes the FEC much wider in application than ARQ. Moreover, the increased decoding complexity of FEC due to the combined need for error detection and correction is no longer a pressing practical issue because the decoder usually lends itself to microprocessor or VLSI implementation in a cost-effective manner.

10.2 *Discrete-Memoryless Channels*

Returning to the model of Figure 10.1*a*, the waveform channel is said to be memoryless if the detector output in a given interval depends only on the signal transmitted in that interval, and not on any previous transmission. Under this condition, we may model the combination of the modulator, the waveform channel, and the detector as a *discrete memoryless channel*. Such a channel is completely described by the set of transition probabilities $p(j|i)$, where i denotes a modulator input symbol, j denotes a demodulator output symbol, and $p(j|i)$ denotes the probability of receiving symbol j, given that symbol i was sent. (Discrete memoryless channels were described previously at some length in Section 9.5.)

The simplest discrete memoryless channel results from the use of binary input and binary output symbols. When binary coding is used, the modulator has only the binary symbols 0 and 1 as inputs. Likewise, the decoder has only binary inputs if binary quantization of the demodulator output is used, that is, a *hard decision* is made on the demodulator output as to which symbol was actually transmitted. In this situation, we have a *binary symmetric channel* (BSC) with a *transition probability diagram* as shown in Figure 10.2. The binary symmetric channel, assuming a channel noise modeled as additive white Gaussian noise (AWGN) channel, is completely described by the *transition probability* p. The majority of coded digital communication systems employ binary coding with hard-decision decoding, due to the simplicity of implementation offered by such an approach. *Hard-decision decoders*, or *algebraic decoders*, take advantage of the special algebraic

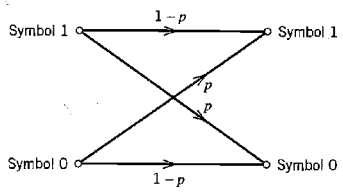


FIGURE 10.2 Transition probability diagram of binary symmetric channel.

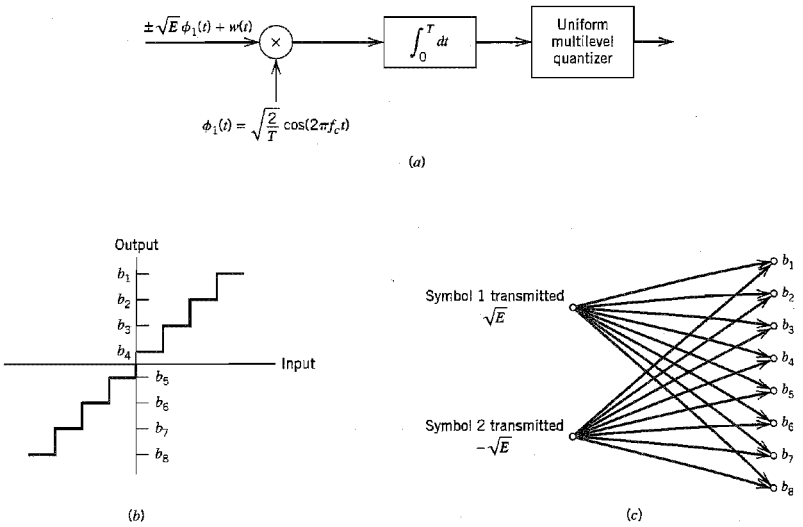


FIGURE 10.3 Binary input Q-ary output discrete memoryless channel. (*a*) Receiver for binary phase-shift keying. (*b*) Transfer characteristic of multilevel quantizer. (*c*) Channel transition probability diagram. Parts (*b*) and (*c*) are illustrated for eight levels of quantization.

structure that is built into the design of channel codes to make the decoding relatively easy to perform.

The use of hard decisions prior to decoding causes an irreversible loss of information in the receiver. To reduce this loss, *soft-decision* coding is used. This is achieved by including a multilevel quantizer at the demodulator output, as illustrated in Figure 10.3*a* for the case of binary PSK signals. The input–output characteristic of the quantizer is shown in Figure 10.3*b*. The modulator has only the binary symbols 0 and 1 as inputs, but the demodulator output now has an alphabet with Q symbols. Assuming the use of the quantizer as described in Figure 10.3*b*, we have $Q = 8$. Such a channel is called a *binary input Q-ary output discrete memoryless channel*. The corresponding channel transition probability diagram is shown in Figure 10.3*c*. The form of this distribution, and consequently the decoder performance, depends on the location of the representation levels of the quantizer, which, in turn, depends on the signal level and noise variance. Accordingly, the demodulator must incorporate automatic gain control if an effective multilevel quantizer is to be realized. Moreover, the use of soft decisions complicates the implementation of the decoder. Nevertheless, soft-decision decoding offers significant improvement in performance over hard-decision decoding by taking a probabilistic rather than an algebraic approach. It is for this reason that soft-decision decoders are also referred to as *probabilistic decoders*.

■ CHANNEL CODING THEOREM REVISITED

In Chapter 9, we established the concept of *channel capacity*, which, for a discrete memoryless channel, represents the maximum amount of information transmitted per

channel use. The *channel coding theorem* states that if a discrete memoryless channel has capacity C and a source generates information at a rate less than C, then there exists a coding technique such that the output of the source may be transmitted over the channel with an arbitrarily low probability of symbol error. For the special case of a binary symmetric channel, the theorem tells us that if the code rate r is less than the channel capacity C, then it is possible to find a code that achieves error-free transmission over the channel. Conversely, it is not possible to find such a code if the code rate r is greater than the channel capacity C.

The channel coding theorem thus specifies the channel capacity C as a *fundamental limit* on the rate at which the transmission of reliable (error-free) messages can take place over a discrete memoryless channel. The issue that matters is not the signal-to-noise ratio, so long as it is large enough, but how the channel input is encoded.

The most unsatisfactory feature of the channel coding theorem, however, is its nonconstructive nature. The theorem asserts the existence of good codes but does not tell us how to find them. By *good codes* we mean families of channel codes that are capable of providing reliable transmission of information (i.e., at arbitrarily small probability of symbol error) over a noisy channel of interest at bit rates up to a maximum value less than the capacity of that channel. The error-control coding techniques described in this chapter provide different methods of designing good codes.

Notation

The codes described in this chapter are *binary codes*, for which the alphabet consists only of symbols 0 and 1. In such a code, the encoding and decoding functions involve the binary arithmetic operations of *modulo-2 addition and multiplication* performed on code words in the code.

Throughout this chapter, we use an ordinary plus sign (+) to denote modulo-2 addition. The use of this terminology will not lead to confusion because the whole chapter relies on binary arithmetic. In so doing, we avoid the use of a special symbol $\oplus$, as we did in preceding chapters. Thus, according to the notation used in this chapter, the rules for modulo-2 addition are as follows:

$$\begin{aligned} 0 + 0 &= 0 \\ 1 + 0 &= 1 \\ 0 + 1 &= 1 \\ 1 + 1 &= 0 \end{aligned}$$

Because $1 + 1 = 0$, it follows that $1 = -1$. Hence, in binary arithmetic, subtraction is the same as addition. The rules for modulo-2 multiplication are as follows:

$$\begin{aligned} 0 \times 0 &= 0 \\ 1 \times 0 &= 0 \\ 0 \times 1 &= 0 \\ 1 \times 1 &= 1 \end{aligned}$$

Division is trivial in that we have

$$\begin{aligned} 1 \div 1 &= 1 \\ 0 \div 1 &= 0 \end{aligned}$$

and division by 0 is not permitted. Modulo-2 addition is the EXCLUSIVE-OR operation in logic, and modulo-2 multiplication is the AND operation.

10.3 *Linear Block Codes*

A code is said to be *linear if any two code words in the code can be added in modulo-2 arithmetic to produce a third code word in the code.* Consider then an (n, k) linear block code, in which k bits of the n code bits are always identical to the message sequence to be transmitted. The $n - k$ bits in the remaining portion are computed from the message bits in accordance with a prescribed encoding rule that determines the mathematical structure of the code. Accordingly, these $n - k$ bits are referred to as *generalized parity check bits* or simply *parity bits*. Block codes in which the message bits are transmitted in unaltered form are called *systematic codes*. For applications requiring *both* error detection and error correction, the use of systematic block codes simplifies implementation of the decoder.

Let $m_0, m_1, \ldots, m_{k-1}$ constitute a block of k arbitrary message bits. Thus we have 2^k distinct message blocks. Let this sequence of message bits be applied to a linear block encoder, producing an n-bit code word whose elements are denoted by $c_0, c_1, \ldots, c_{n-1}$. Let $b_0, b_1, \ldots, b_{n-k-1}$ denote the $(n - k)$ parity bits in the code word. For the code to possess a systematic structure, a code word is divided into two parts, one of which is occupied by the message bits and the other by the parity bits. Clearly, we have the option of sending the message bits of a code word before the parity bits, or vice versa. The former option is illustrated in Figure 10.4, and its use is assumed in the sequel.

According to the representation of Figure 10.4, the $(n - k)$ left-most bits of a code word are identical to the corresponding parity bits, and the k right-most bits of the code word are identical to the corresponding message bits. We may therefore write

$$c_i = \begin{cases} b_i, & i = 0, 1, \ldots, n - k - 1 \\ m_{i+k-n}, & i = n - k, n - k + 1, \ldots, n - 1 \end{cases} \tag{10.1}$$

The $(n - k)$ parity bits are *linear sums* of the k message bits, as shown by the generalized relation

$$b_i = p_{0i}m_0 + p_{1i}m_1 + \cdots + p_{k-1,i}m_{k-1} \tag{10.2}$$

where the coefficients are defined as follows:

$$p_{ij} = \begin{cases} 1 & \text{if } b_i \text{ depends on } m_j \\ 0 & \text{otherwise} \end{cases} \tag{10.3}$$

The coefficients p_{ij} are chosen in such a way that the rows of the generator matrix are linearly independent and the parity equations are *unique*.

The system of Equations (10.1) and (10.2) defines the mathematical structure of the (n, k) linear block code. This system of equations may be rewritten in a compact form

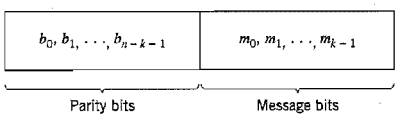


FIGURE 10.4 Structure of systematic code word.

using matrix notation. To proceed with this reformulation, we define the 1-by-k *message vector*, or *information vector*, $\mathbf{m}$, the 1-by-$(n-k)$ parity vector $\mathbf{b}$, and the 1-by-n code vector $\mathbf{c}$ as follows:

$$\mathbf{m} = [m_0, m_1, \ldots, m_{k-1}] \tag{10.4}$$

$$\mathbf{b} = [b_0, b_1, \ldots, b_{n-k-1}] \tag{10.5}$$

$$\mathbf{c} = [c_0, c_1, \ldots, c_{n-1}] \tag{10.6}$$

Note that all three vectors are *row vectors*. The use of row vectors is adopted in this chapter for the sake of being consistent with the notation commonly used in the coding literature. We may thus rewrite the set of simultaneous equations defining the parity bits in the compact matrix form:

$$\mathbf{b} = \mathbf{mP} \tag{10.7}$$

where $\mathbf{P}$ is the k-by-$(n-k)$ *coefficient matrix* defined by

$$\mathbf{P} = \begin{bmatrix} p_{00} & p_{01} & \cdots & p_{0,n-k-1} \\ p_{10} & p_{11} & \cdots & p_{1,n-k-1} \\ \vdots & \vdots & & \vdots \\ p_{k-1,0} & p_{k-1,1} & \cdots & p_{k-1,n-k-1} \end{bmatrix} \tag{10.8}$$

where p_{ij} is 0 or 1.

From the definitions given in Equations (10.4)–(10.6), we see that $\mathbf{c}$ may be expressed as a partitioned row vector in terms of the vectors $\mathbf{m}$ and $\mathbf{b}$ as follows:

$$\mathbf{c} = [\mathbf{b} : \mathbf{m}] \tag{10.9}$$

Hence, substituting Equation (10.7) into Equation (10.9) and factoring out the common message vector $\mathbf{m}$, we get

$$\mathbf{c} = \mathbf{m}[\mathbf{P} : \mathbf{I}_k] \tag{10.10}$$

where $\mathbf{I}_k$ is the k-by-k *identity matrix*:

$$\mathbf{I}_k = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix} \tag{10.11}$$

Define the k-by-n *generator matrix*

$$\mathbf{G} = [\mathbf{P} : \mathbf{I}_k] \tag{10.12}$$

The generator matrix $\mathbf{G}$ of Equation (10.12) is said to be in the *canonical form* in that its k rows are linearly independent; that is, it is not possible to express any row of the matrix $\mathbf{G}$ as a linear combination of the remaining rows. Using the definition of the generator matrix $\mathbf{G}$, we may simplify Equation (10.10) as

$$\mathbf{c} = \mathbf{mG} \tag{10.13}$$

The full set of code words, referred to simply as *the code*, is generated in accordance with Equation (10.13) by letting the message vector $\mathbf{m}$ range through the set of all 2^k binary k-tuples (1-by-k vectors). Moreover, the sum of any two code words is another

code word. This basic property of linear block codes is called *closure*. To prove its validity, consider a pair of code vectors $\mathbf{c}_i$ and $\mathbf{c}_j$ corresponding to a pair of message vectors $\mathbf{m}_i$ and $\mathbf{m}_j$, respectively. Using Equation (10.13) we may express the sum of $\mathbf{c}_i$ and $\mathbf{c}_j$ as

$$\begin{aligned}\mathbf{c}_i + \mathbf{c}_j &= \mathbf{m}_i\mathbf{G} + \mathbf{m}_j\mathbf{G} \\ &= (\mathbf{m}_i + \mathbf{m}_j)\mathbf{G}\end{aligned}$$

The modulo-2 sum of $\mathbf{m}_i$ and $\mathbf{m}_j$ represents a new message vector. Correspondingly, the modulo-2 sum of $\mathbf{c}_i$ and $\mathbf{c}_j$ represents a new code vector.

There is another way of expressing the relationship between the message bits and parity-check bits of a linear block code. Let $\mathbf{H}$ denote an $(n - k)$-by-n matrix, defined as

$$\mathbf{H} = [\mathbf{I}_{n-k} : \mathbf{P}^T] \tag{10.14}$$

where $\mathbf{P}^T$ is an $(n - k)$-by-k matrix, representing the transpose of the coefficient matrix $\mathbf{P}$, and $\mathbf{I}_{n-k}$ is the $(n - k)$-by-$(n - k)$ identity matrix. Accordingly, we may perform the following multiplication of partitioned matrices:

$$\begin{aligned}\mathbf{H}\mathbf{G}^T &= [\mathbf{I}_{n-k} : \mathbf{P}^T]\begin{bmatrix}\mathbf{P}^T \\ \cdots \\ \mathbf{I}_k\end{bmatrix} \\ &= \mathbf{P}^T + \mathbf{P}^T\end{aligned}$$

where we have used the fact that multiplication of a rectangular matrix by an identity matrix of compatible dimensions leaves the matrix unchanged. In modulo-2 arithmetic, we have $\mathbf{P}^T + \mathbf{P}^T = 0$, where 0 denotes an $(n - k)$-by-k null matrix (i.e., a matrix that has zeros for all of its elements). Hence,

$$\mathbf{H}\mathbf{G}^T = 0 \tag{10.15}$$

Equivalently, we have $\mathbf{G}\mathbf{H}^T = 0$, where 0 is a new null matrix. Postmultiplying both sides of Equation (10.13) by $\mathbf{H}^T$, the transpose of $\mathbf{H}$, and then using Equation (10.15), we get

$$\begin{aligned}\mathbf{c}\mathbf{H}^T &= \mathbf{m}\mathbf{G}\mathbf{H}^T \\ &= 0\end{aligned} \tag{10.16}$$

The matrix $\mathbf{H}$ is called the *parity-check matrix* of the code, and the set of equations specified by Equation (10.16) are called *parity-check equations*.

The generator equation (10.13) and the parity-check detector equation (10.16) are basic to the description and operation of a linear block code. These two equations are depicted in the form of block diagrams in Figure 10.5*a* and 10.5*b*, respectively.

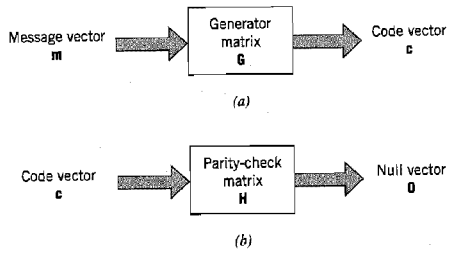


FIGURE 10.5 Block diagram representations of the generator equation (10.13) and the parity-check equation (10.16).

EXAMPLE 10.1 Repetition Codes

Repetition codes represent the simplest type of linear block codes. In particular, a single message bit is encoded into a block of n identical bits, producing an $(n, 1)$ block code. Such a code allows provision for a variable amount of redundancy. There are only two code words in the code: an all-zero code word and an all-one code word.

Consider, for example, the case of a repetition code with $k = 1$ and $n = 5$. In this case, we have four parity bits that are the same as the message bit. Hence, the identity matrix $\mathbf{I}_k = 1$, and the coefficient matrix P consists of a 1-by-4 vector that has 1 for all of its elements. Correspondingly, the generator matrix equals a row vector of all 1s, as shown by

$$\mathbf{G} = [1 \quad 1 \quad 1 \quad 1 \vdots 1]$$

The transpose of the coefficient matrix P, namely, matrix $\mathbf{P}^T$, consists of a 4-by-1 vector that has 1 for all of its elements. The identity matrix $\mathbf{I}_{n-k}$ consists of a 4-by-4 matrix. Hence, the parity-check matrix equals

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \vdots 1 \\ 0 & 1 & 0 & 0 \vdots 1 \\ 0 & 0 & 1 & 0 \vdots 1 \\ 0 & 0 & 0 & 1 \vdots 1 \end{bmatrix}$$

Since the message vector consists of a single binary symbol, 0 or 1, it follows from Equation (10.13) that there are only two code words: 00000 and 11111 in the (5, 1) repetition code, as expected. Note also that $\mathbf{HG}^T = 0$, modulo-2, in accordance with Equation (10.15). ◀

SYNDROME: DEFINITION AND PROPERTIES

The generator matrix G is used in the encoding operation at the transmitter. On the other hand, the parity-check matrix H is used in the decoding operation at the receiver. In the context of the latter operation, let r denote the 1-by-n *received vector* that results from sending the code vector c over a noisy channel. We express the vector r as the sum of the original code vector c and a vector e, as shown by

$$\mathbf{r} = \mathbf{c} + \mathbf{e} \tag{10.17}$$

The vector e is called the *error vector* or *error pattern*. The ith element of e equals 0 if the corresponding element of r is the same as that of c. On the other hand, the ith element of e equals 1 if the corresponding element of r is different from that of c, in which case an error is said to have occurred in the ith location. That is, for $i = 1, 2, \ldots, n$, we have

$$e_i = \begin{cases} 1 & \text{if an error has occurred in the } i\text{th location} \\ 0 & \text{otherwise} \end{cases} \tag{10.18}$$

The receiver has the task of decoding the code vector c from the received vector r. The algorithm commonly used to perform this decoding operation starts with the computation of a 1-by-$(n - k)$ vector called the *error-syndrome vector* or simply the *syndrome*.[3] The importance of the syndrome lies in the fact that it depends only upon the error pattern.

Given a 1-by-n received vector r, the corresponding syndrome is formally defined as

$$\mathbf{s} = \mathbf{rH}^T \tag{10.19}$$

Accordingly, the syndrome has the following important properties.

Property 1

The syndrome depends only on the error pattern, and not on the transmitted code word.

To prove this property, we first use Equations (10.17) and (10.19), and then Equation (10.16) to obtain

$$\begin{aligned} \mathbf{s} &= (\mathbf{c} + \mathbf{e})\mathbf{H}^T \\ &= \mathbf{c}\mathbf{H}^T + \mathbf{e}\mathbf{H}^T \\ &= \mathbf{e}\mathbf{H}^T \end{aligned} \tag{10.20}$$

Hence, the parity-check matrix H of a code permits us to compute the syndrome s, which depends only upon the error pattern e.

Property 2

All error patterns that differ by a code word have the same syndrome.

For k message bits, there are 2^k distinct code vectors denoted as $\mathbf{c}_i$, $i = 0, 1, \ldots, 2^k - 1$. Correspondingly, for any error pattern e, we define the 2^k distinct vectors $\mathbf{e}_i$ as

$$\mathbf{e}_i = \mathbf{e} + \mathbf{c}_i, \qquad i = 0, 1, \ldots, 2^k - 1 \tag{10.21}$$

The set of vectors $\{\mathbf{e}_i, i = 0, 1, \ldots, 2^k - 1\}$ so defined is called a *coset* of the code. In other words, a coset has exactly 2^k elements that differ at most by a code vector. Thus, an (n, k) linear block code has 2^{n-k} possible cosets. In any event, multiplying both sides of Equation (10.21) by the matrix $\mathbf{H}^T$, we get

$$\begin{aligned} \mathbf{e}_i\mathbf{H}^T &= \mathbf{e}\mathbf{H}^T + \mathbf{c}_i\mathbf{H}^T \\ &= \mathbf{e}\mathbf{H}^T \end{aligned} \tag{10.22}$$

which is independent of the index i. Accordingly, we may state that each coset of the code is characterized by a unique syndrome.

We may put Properties 1 and 2 in perspective by expanding Equation (10.20). Specifically, with the matrix H having the systematic form given in Equation (10.14), where the matrix P is itself defined by Equation (10.8), we find from Equation (10.20) that the $(n - k)$ elements of the syndrome s are linear combinations of the n elements of the error pattern e, as shown by

$$\begin{aligned} s_0 &= e_0 + e_{n-k}p_{00} + e_{n-k+1}p_{10} + \cdots + e_{n-1}p_{k-1,0} \\ s_1 &= e_1 + e_{n-k}p_{01} + e_{n-k+1}p_{11} + \cdots + e_{n-1}p_{k-1,1} \\ &\vdots \\ s_{n-k-1} &= e_{n-k-1} + e_{n-k}p_{0,n-k-1} + \cdots + e_{n-1}p_{k-1,n-k-1} \end{aligned} \tag{10.23}$$

This set of $(n - k)$ linear equations clearly shows that the syndrome contains information about the error pattern and may therefore be used for error detection. However, it should be noted that the set of equations is *underdetermined* in that we have more unknowns than equations. Accordingly, there is *no* unique solution for the error pattern. Rather, there are 2^n error patterns that satisfy Equation (10.23) and therefore result in the same syndrome, in accordance with Property 2 and Equation (10.22). In particular, with 2^{n-k} possible syndrome vectors, the information contained in the syndrome s about the error pattern e is *not* enough for the decoder to compute the exact value of the transmitted code vector. Nevertheless, knowledge of the syndrome s reduces the search for the true error

pattern e from 2^n to 2^{n-k} possibilities. Given these possibilities, the decoder has the task of making the best selection from the cosets corresponding to s.

Minimum Distance Considerations

Consider a pair of code vectors $\mathbf{c}_1$ and $\mathbf{c}_2$ that have the same number of elements. The *Hamming distance* $d(\mathbf{c}_1, \mathbf{c}_2)$ between such a pair of code vectors is defined as the number of locations in which their respective elements differ.

The *Hamming weight* $w(\mathbf{c})$ of a code vector c is defined as the number of nonzero elements in the code vector. Equivalently, we may state that the Hamming weight of a code vector is the distance between the code vector and the all-zero code vector.

The *minimum distance* $d_{\min}$ of a linear block code is defined as the smallest Hamming distance between any pair of code vectors in the code. That is, the minimum distance is the same as the smallest Hamming weight of the difference between any pair of code vectors. From the closure property of linear block codes, the sum (or difference) of two code vectors is another code vector. Accordingly, we may state that *the minimum distance of a linear block code is the smallest Hamming weight of the nonzero code vectors in the code.*

The minimum distance $d_{\min}$ is related to the structure of the parity-check matrix H of the code in a fundamental way. From Equation (10.16) we know that a linear block code is defined by the set of all code vectors for which $\mathbf{c}\mathbf{H}^T = 0$, where $\mathbf{H}^T$ is the transpose of the parity-check matrix H. Let the matrix H be expressed in terms of its columns as follows:

$$\mathbf{H} = [\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_n] \tag{10.24}$$

Then, for a code vector c to satisfy the condition $\mathbf{c}\mathbf{H}^T = 0$, the vector c must have 1s in such positions that the corresponding rows of $\mathbf{H}^T$ sum to the zero vector 0. However, by definition, the number of 1s in a code vector is the Hamming weight of the code vector. Moreover, the smallest Hamming weight of the nonzero code vectors in a linear block code equals the minimum distance of the code. Hence, *the minimum distance of a linear block code is defined by the minimum number of rows of the matrix* $\mathbf{H}^T$ whose sum is equal to the zero vector.

The minimum distance of a linear block code, $d_{\min}$, is an important parameter of the code. Specifically, it determines the error-correcting capability of the code. Suppose an (n, k) linear block code is required to detect and correct all error patterns (over a binary symmetric channel), and whose Hamming weight is less than or equal to t. That is, if a code vector $\mathbf{c}_i$ in the code is transmitted and the received vector is $\mathbf{r} = \mathbf{c}_i + \mathbf{e}$, we require that the decoder output $\hat{\mathbf{c}} = \mathbf{c}_i$, whenever the error pattern e has a Hamming weight $w(\mathbf{e}) \leq t$. We assume that the 2^k code vectors in the code are transmitted with equal probability. The best strategy for the decoder then is to pick the code vector closest to the received vector r, that is, the one for which the Hamming distance $d(\mathbf{c}_i, \mathbf{r})$ is the smallest. With such a strategy, the decoder will be able to detect and correct all error patterns of Hamming weight $w(\mathbf{e}) \leq t$, provided that the minimum distance of the code is equal to or greater than $2t + 1$. We may demonstrate the validity of this requirement by adopting a geometric interpretation of the problem. In particular, the 1-by-n code vectors and the 1-by-n received vector are represented as points in an n-dimensional space. Suppose that we construct two spheres, each of radius t, around the points that represent code vectors $\mathbf{c}_i$ and $\mathbf{c}_j$. Let these two spheres be disjoint, as depicted in Figure 10.6*a*. For this condition to be satisfied, we require that $d(\mathbf{c}_i, \mathbf{c}_j) \geq 2t + 1$. If then the code vector $\mathbf{c}_i$ is transmitted and the Hamming distance $d(\mathbf{c}_i, \mathbf{r}) \leq t$, it is clear that the decoder will pick $\mathbf{c}_i$ as it is the

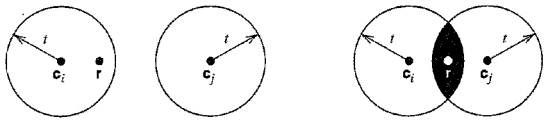


FIGURE 10.6 (*a*) Hamming distance $d(\mathbf{c}_i, \mathbf{c}_j) \geq 2t + 1$. (*b*) Hamming distance $d(\mathbf{c}_i, \mathbf{c}_j) < 2t$. The received vector is denoted by r.

code vector closest to the received vector r. If, on the other hand, the Hamming distance $d(\mathbf{c}_i, \mathbf{c}_j) \leq 2t$, the two spheres around $\mathbf{c}_i$ and $\mathbf{c}_j$ intersect, as depicted in Figure 10.6*b*. Here we see that if $\mathbf{c}_i$ is transmitted, there exists a received vector r such that the Hamming distance $d(\mathbf{c}_i, \mathbf{r}) \leq t$, and yet r is as close to $\mathbf{c}_j$ as it is to $\mathbf{c}_i$. Clearly, there is now the possibility of the decoder picking the vector $\mathbf{c}_j$, which is wrong. We thus conclude that *an* (n, k) *linear block code has the power to correct all error patterns of weight* t *or less if, and only if,*

$$d(\mathbf{c}_i, \mathbf{c}_j) \geq 2t + 1 \qquad \text{for all } \mathbf{c}_i \text{ and } \mathbf{c}_j$$

By definition, however, the smallest distance between any pair of code vectors in a code is the minimum distance of the code, $d_{\min}$. We may therefore state that *an* (n, k) *linear block code of minimum distance* $d_{\min}$ *can correct up to* t *errors if, and only if,*

$$t \leq \lfloor \tfrac{1}{2}(d_{\min} - 1) \rfloor \tag{10.25}$$

where $\lfloor\ \rfloor$ *denotes the largest integer* less than or equal to the enclosed quantity. Equation (10.25) gives the error-correcting capability of a linear block code a quantitative meaning.

■ SYNDROME DECODING

We are now ready to describe a syndrome-based decoding scheme for linear block codes. Let $\mathbf{c}_1, \mathbf{c}_2, \ldots, \mathbf{c}_{2^k}$ denote the 2^k code vectors of an (n, k) linear block code. Let r denote the received vector, which may have one of 2^n possible values. The receiver has the task of partitioning the 2^n possible received vectors into 2^k disjoint subsets $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_{2^k}$ in such a way that the ith subset $\mathcal{D}_i$ corresponds to code vector $\mathbf{c}_i$ for $1 \leq i \leq 2^k$. The received vector r is decoded into $\mathbf{c}_i$ if it is in the ith subset. For the decoding to be correct, r must be in the subset that belongs to the code vector $\mathbf{c}_i$ that was actually sent.

The 2^k subsets described herein constitute a *standard array* of the linear block code. To construct it, we may exploit the linear structure of the code by proceeding as follows:

1. The 2^k code vectors are placed in a row with the all-zero code vector $\mathbf{c}_1$ as the left-most element.
2. An error pattern $\mathbf{e}_2$ is picked and placed under $\mathbf{c}_1$, and a second row is formed by adding $\mathbf{e}_2$ to each of the remaining code vectors in the first row; it is important that the error pattern chosen as the first element in a row not have previously appeared in the standard array.
3. Step 2 is repeated until all the possible error patterns have been accounted for.

Figure 10.7 illustrates the structure of the standard array so constructed. The 2^k columns of this array represent the disjoint subsets $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_{2^k}$. The 2^{n-k} rows of the array

$$
\begin{array}{ccccccc}
\mathbf{c}_1 = \mathbf{0} & \mathbf{c}_2 & \mathbf{c}_3 & \cdots & \mathbf{c}_i & \cdots & \mathbf{c}_{2^k} \\
\mathbf{e}_2 & \mathbf{c}_2 + \mathbf{e}_2 & \mathbf{c}_3 + \mathbf{e}_2 & \cdots & \mathbf{c}_i + \mathbf{e}_2 & \cdots & \mathbf{c}_{2^k} + \mathbf{e}_2 \\
\mathbf{e}_3 & \mathbf{c}_2 + \mathbf{e}_3 & \mathbf{c}_3 + \mathbf{e}_3 & \cdots & \mathbf{c}_i + \mathbf{e}_3 & \cdots & \mathbf{c}_{2^k} + \mathbf{e}_3 \\
\vdots & \vdots & \vdots & & \vdots & & \vdots \\
\mathbf{e}_j & \mathbf{c}_2 + \mathbf{e}_j & \mathbf{c}_3 + \mathbf{e}_j & \cdots & \mathbf{c}_i + \mathbf{e}_j & \cdots & \mathbf{c}_{2^k} + \mathbf{e}_j \\
\vdots & \vdots & \vdots & & \vdots & & \vdots \\
\mathbf{e}_{2^{n-k}} & \mathbf{c}_2 + \mathbf{e}_{2^{n-k}} & \mathbf{c}_3 + \mathbf{e}_{2^{n-k}} & & \mathbf{c}_i + \mathbf{e}_{2^{n-k}} & & \mathbf{c}_{2^k} + \mathbf{e}_{2^{n-k}}
\end{array}
$$

FIGURE 10.7 Standard array for an (n, k) block code.

represent the cosets of the code, and their first elements $\mathbf{e}_2, \ldots, \mathbf{e}_{2^{n-k}}$ are called *coset leaders*.

For a given channel, the probability of decoding error is minimized when the most likely error patterns (i.e., those with the largest probability of occurrence) are chosen as the coset leaders. In the case of a binary symmetric channel, the smaller the Hamming weight of an error pattern the more likely it is to occur. Accordingly, the standard array should be constructed with each coset leader having the minimum Hamming weight in its coset.

We may now describe a decoding procedure for a linear block code:

1. For the received vector $\mathbf{r}$, compute the syndrome $\mathbf{s} = \mathbf{r}\mathbf{H}^T$.
2. Within the coset characterized by the syndrome $\mathbf{s}$, identify the coset leader (i.e., the error pattern with the largest probability of occurrence); call it $\mathbf{e}_0$.
3. Compute the code vector

$$\mathbf{c} = \mathbf{r} + \mathbf{e}_0 \tag{10.26}$$

as the decoded version of the received vector $\mathbf{r}$.

This procedure is called *syndrome decoding*.

▶ EXAMPLE 10.2 Hamming Codes[4]

Consider a family of (n, k) linear block codes that have the following parameters:

Block length:	$n = 2^m - 1$
Number of message bits:	$k = 2^m - m - 1$
Number of parity bits:	$n - k = m$

where $m \geq 3$. These are the so-called Hamming codes.

Consider, for example, the (7, 4) Hamming code with $n = 7$ and $k = 4$, corresponding to $m = 3$. The generator matrix of the code must have a structure that conforms to Equation (10.12). The following matrix represents an appropriate generator matrix for the (7, 4) Hamming code:

$$\mathbf{G} = \left[\begin{array}{ccc:cccc} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 \end{array}\right] = [\underbrace{\mathbf{P}}_{} : \underbrace{\mathbf{I}_k}_{}]$$

TABLE 10.1 *Code words of a (7, 4) Hamming code*

Message Word	*Code Word*	*Weight of Code Word*	*Message Word*	*Code Word*	*Weight of Code Word*
0 0 0 0	0 0 0 0 0 0 0	0	1 0 0 0	1 1 0 1 0 0 0	3
0 0 0 1	1 0 1 0 0 0 1	3	1 0 0 1	0 1 1 1 0 0 1	4
0 0 1 0	1 1 1 0 0 1 0	4	1 0 1 0	0 0 1 1 0 1 0	3
0 0 1 1	0 1 0 0 0 1 1	3	1 0 1 1	1 0 0 1 0 1 1	4
0 1 0 0	0 1 1 0 1 0 0	3	1 1 0 0	1 0 1 1 1 0 0	4
0 1 0 1	1 1 0 0 1 0 1	4	1 1 0 1	0 0 0 1 1 0 1	3
0 1 1 0	1 0 0 0 1 1 0	3	1 1 1 0	0 1 0 1 1 1 0	4
0 1 1 1	0 0 1 0 1 1 1	4	1 1 1 1	1 1 1 1 1 1 1	7

The corresponding parity-check matrix is given by

$$\mathbf{H} = \left[\begin{array}{cccc:cccc} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{array}\right]$$

$$\underbrace{\mathbf{I}_{n-k}}_{} \qquad \underbrace{\mathbf{P}^T}_{}$$

With $k = 4$, there are $2^k = 16$ distinct message words, which are listed in Table 10.1. For a given message word, the corresponding code word is obtained by using Equation (10.13). Thus, the application of this equation results in the 16 code words listed in Table 10.1.

In Table 10.1, we have also listed the Hamming weights of the individual code words in the (7, 4) Hamming code. Since the smallest of the Hamming weights for the nonzero code words is 3, it follows that the minimum distance of the code is 3. Indeed, Hamming codes have the property that the minimum distance $d_{\min} = 3$, independent of the value assigned to the number of parity bits m.

To illustrate the relation between the minimum distance $d_{\min}$ and the structure of the parity-check matrix H, consider the code word 0110100. In the matrix multiplication defined by Equation (10.16), the nonzero elements of this code word "sift" out the second, third, and fifth columns of the matrix H yielding

$$\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

We may perform similar calculations for the remaining 14 nonzero code words. We thus find that the smallest number of columns in H that sums to zero is 3, confirming the earlier statement that $d_{\min} = 3$.

An important property of Hamming codes is that they satisfy the condition of Equation (10.25) with the equality sign, assuming that $t = 1$. This means that Hamming codes are *single-error correcting binary perfect codes.*

Assuming single-error patterns, we may formulate the seven coset leaders listed in the right-hand column of Table 10.2. The corresponding 2^3 syndromes, listed in the left-hand column, are calculated in accordance with Equation (10.20). The zero syndrome signifies no transmission errors.

Suppose, for example, the code vector [1110010] is sent, and the received vector is

TABLE 10.2 *Decoding table for the (7, 4) Hamming code defined in Table 10.1*

Syndrome	*Error Pattern*
0 0 0	0 0 0 0 0 0 0
1 0 0	1 0 0 0 0 0 0
0 1 0	0 1 0 0 0 0 0
0 0 1	0 0 1 0 0 0 0
1 1 0	0 0 0 1 0 0 0
0 1 1	0 0 0 0 1 0 0
1 1 1	0 0 0 0 0 1 0
1 0 1	0 0 0 0 0 0 1

[1100010] with an error in the third bit. Using Equation (10.19), the syndrome is calculated to be

$$\mathbf{s} = [1100010]\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

$$= [0 \quad 0 \quad 1]$$

From Table 10.2 the corresponding coset leader (i.e., error pattern with the highest probability of occurrence) is found to be [0010000], indicating correctly that the third bit of the received vector is erroneous. Thus, adding this error pattern to the received vector, in accordance with Equation (10.26), yields the correct code vector actually sent. ◀

DUAL CODE

Given a linear block code, we may define its *dual* as follows. Taking the transpose of both sides of Equation (10.15), we have

$$\mathbf{G}\mathbf{H}^T = \mathbf{0}$$

where $\mathbf{H}^T$ is the transpose of the parity-check matrix of the code, and $\mathbf{0}$ is a new zero matrix. This equation suggests that every (n, k) linear block code with generator matrix $\mathbf{G}$ and parity-check matrix $\mathbf{H}$ has a *dual code* with parameters $(n, n - k)$, generator matrix $\mathbf{H}$ and parity-check matrix $\mathbf{G}$.

10.4 Cyclic Codes

Cyclic codes form a subclass of linear block codes. Indeed, many of the important linear block codes discovered to date are either cyclic codes or closely related to cyclic codes. An

advantage of cyclic codes over most other types of codes is that they are easy to encode. Furthermore, cyclic codes possess a well-defined mathematical structure, which has led to the development of very efficient decoding schemes for them.

A binary code is said to be a *cyclic code* if it exhibits two fundamental properties:

1. *Linearity property:* The sum of any two code words in the code is also a code word.
2. *Cyclic property:* Any cyclic shift of a code word in the code is also a code word.

Property 1 restates the fact that a cyclic code is a linear block code (i.e., it can be described as a parity-check code). To restate Property 2 in mathematical terms, let the n-tuple $(c_0, c_1, \ldots, c_{n-1})$ denote a code word of an (n, k) linear block code. The code is a cyclic code if the n-tuples

$$\begin{aligned}&(c_{n-1}, c_0, \ldots, c_{n-2}),\\ &(c_{n-2}, c_{n-1}, \ldots, c_{n-3}),\\ &\vdots\\ &(c_1, c_2, \ldots, c_{n-1}, c_0)\end{aligned}$$

are all code words in the code.

To develop the algebraic properties of cyclic codes, we use the elements $c_0, c_1, \ldots, c_{n-1}$ of a code word to define the *code polynomial*

$$c(X) = c_0 + c_1X + c_2X^2 + \cdots + c_{n-1}X^{n-1} \tag{10.27}$$

where X is an indeterminate. Naturally, for binary codes, the coefficients are 1s and 0s. Each power of X in the polynomial $c(X)$ represents a one-bit *shift* in time. Hence, multiplication of the polynomial $c(X)$ by X may be viewed as a shift to the right. The key question is: How do we make such a shift *cyclic*? The answer to this question is addressed next.

Let the code polynomial $c(X)$ be multiplied by X^i, yielding

$$\begin{aligned}X^ic(X) &= X^i(c_0 + c_1X + \cdots + c_{n-i-1}X^{n-i-1} + c_{n-i}X^{n-i}\\ &\quad + \cdots + c_{n-1}X^{n-1})\\ &= c_0X^i + c_1X^{i+1} + \cdots + c_{n-i-1}X^{n-1} + c_{n-i}X^n\\ &\quad + \cdots + c_{n-1}X^{n+i-1}\\ &= c_{n-i}X^n + \cdots + c_{n-1}X^{n+i-1} + c_0X^i + c_1X^{i+1}\\ &\quad + \cdots + c_{n-i-1}X^{n-1}\end{aligned} \tag{10.28}$$

where, in the last line, we have merely rearranged terms. Recognizing, for example, that $c_{n-i} + c_{n-i} = 0$ in modulo-2 addition, we may manipulate the first i terms of Equation (10.28) as follows:

$$\begin{aligned}X^ic(X) &= c_{n-i} + \cdots + c_{n-1}X^{i-1} + c_0X^i + c_1X^{i+1} + \cdots + c_{n-i-1}X^{n-1}\\ &\quad + c_{n-i}(X^n + 1) + \cdots + c_{n-1}X^{i-1}(X^n + 1)\end{aligned} \tag{10.29}$$

Next, we introduce the following definitions:

$$\begin{aligned}c^{(i)}(X) &= c_{n-i} + \cdots + c_{n-1}X^{i-1} + c_0X^i + c_1X^{i+1}\\ &\quad + \cdots + c_{n-i-1}X^{n-1}\end{aligned} \tag{10.30}$$

$$q(X) = c_{n-i} + c_{n-i+1}X + \cdots + c_{n-1}X^{i-1} \tag{10.31}$$

Accordingly, Equation (10.29) is reformulated in the compact form

$$X^i c(X) = q(X)(X^n + 1) + c^{(i)}(X) \tag{10.32}$$

The polynomial $c^{(i)}(X)$ is recognized as the code polynomial of the code word $(c_{n-i}, \ldots, c_{n-1}, c_0, c_1, \ldots, c_{n-i-1})$ obtained by applying i cyclic shifts to the code word $(c_0, c_1, \ldots, c_{n-i-1}, c_{n-i}, \ldots, c_{n-1})$. Moreover, from Equation (10.32) we readily see that $c^{(i)}(X)$ is the remainder that results from dividing $X^i c(X)$ by $(X^n + 1)$. We may thus formally state the cyclic property in polynomial notation as follows: *If $c(X)$ is a code polynomial, then the polynomial*

$$c^{(i)}(X) = X^i c(X) \bmod (X^n + 1) \tag{10.33}$$

is also a code polynomial for any cyclic shift i; the term *mod* is the abbreviation for *modulo*. The special form of polynomial multiplication described in Equation (10.33) is referred to as *multiplication modulo* $X^n + 1$. In effect, the multiplication is subject to the constraint $X^n = 1$, the application of which restores the polynomial $X^i c(X)$ to order $n - 1$ for all $i < n$. (Note that in modulo-2 arithmetic, $X^n + 1$ has the same value as $X^n - 1$.)

Generator Polynomial

The polynomial $X^n + 1$ and its factors play a major role in the generation of cyclic codes. Let $g(X)$ be a polynomial of degree $n - k$ that is a factor of $X^n + 1$; as such, $g(X)$ is *the polynomial of least degree in the code*. In general, $g(X)$ may be expanded as follows:

$$g(X) = 1 + \sum_{i=1}^{n-k-1} g_i X^i + X^{n-k} \tag{10.34}$$

where the coefficient g_i is equal to 0 or 1. According to this expansion, the polynomial $g(X)$ has two terms with coefficient 1 separated by $n - k - 1$ terms. The polynomial $g(X)$ is called the *generator polynomial* of a cyclic code. A cyclic code is uniquely determined by the generator polynomial $g(X)$ in that each code polynomial in the code can be expressed in the form of a polynomial product as follows:

$$c(X) = a(X)g(X) \tag{10.35}$$

where $a(X)$ is a polynomial in X with degree $k - 1$. The $c(X)$ so formed satisfies the condition of Equation (10.33) since $g(X)$ is a factor of $X^n + 1$.

Suppose we are given the generator polynomial $g(X)$ and the requirement is to encode the message sequence $(m_0, m_1, \ldots, m_{k-1})$ into an (n, k) *systematic* cyclic code. That is, the message bits are transmitted in unaltered form, as shown by the following structure for a code word (see Figure 10.4):

$$\underbrace{(b_0, b_1, \ldots, b_{n-k-1},}_{n-k \text{ parity bits}} \quad \underbrace{m_0, m_1, \ldots, m_{k-1})}_{k \text{ message bits}}$$

Let the *message polynomial* be defined by

$$m(X) = m_0 + m_1 X + \cdots + m_{k-1} X^{k-1} \tag{10.36}$$

and let

$$b(X) = b_0 + b_1 X + \cdots + b_{n-k-1} X^{n-k-1} \tag{10.37}$$

According to Equation (10.1), we want the code polynomial to be in the form

$$c(X) = b(X) + X^{n-k}m(X) \tag{10.38}$$

Hence, the use of Equations (10.35) and (10.38) yields

$$a(X)g(X) = b(X) + X^{n-k}m(X)$$

Equivalently, in light of modulo-2 addition, we may write

$$\frac{X^{n-k}m(X)}{g(X)} = a(X) + \frac{b(X)}{g(X)} \tag{10.39}$$

Equation (10.39) states that the polynomial $b(X)$ is the *remainder* left over after dividing $X^{n-k}m(X)$ by $g(X)$.

We may now summarize the steps involved in the encoding procedure for an (n, k) cyclic code assured of a systematic structure. Specifically, we proceed as follows:

1. Multiply the message polynomial $m(X)$ by X^{n-k}.
2. Divide $X^{n-k}m(X)$ by the generator polynomial $g(X)$, obtaining the remainder $b(X)$.
3. Add $b(X)$ to $X^{n-k}m(X)$, obtaining the code polynomial $c(X)$.

■ PARITY-CHECK POLYNOMIAL

An (n, k) cyclic code is uniquely specified by its generator polynomial $g(X)$ of order $(n - k)$. Such a code is also uniquely specified by another polynomial of degree k, which is called the *parity-check polynomial*, defined by

$$h(X) = 1 + \sum_{i=1}^{k-1} h_i X^i + X^k \tag{10.40}$$

where the coefficients h_i are 0 or 1. The parity-check polynomial $h(X)$ has a form similar to the generator polynomial in that there are two terms with coefficient 1, but separated by $k - 1$ terms.

The generator polynomial $g(X)$ is equivalent to the generator matrix $\mathbf{G}$ as a description of the code. Correspondingly, the parity-check polynomial, denoted by $h(X)$, is an equivalent representation of the parity-check matrix $\mathbf{H}$. We thus find that the matrix relation $\mathbf{HG}^T = 0$ presented in Equation (10.15) for linear block codes corresponds to the relationship

$$g(X)h(X) \bmod (X^n + 1) = 0 \tag{10.41}$$

Accordingly, we may state that *the generator polynomial $g(X)$ and the parity-check polynomial $h(X)$ are factors of the polynomial* $X^n + 1$, as shown by

$$g(X)h(X) = X^n + 1 \tag{10.42}$$

This property provides the basis for selecting the generator or parity-check polynomial of a cyclic code. In particular, we may state that if $g(X)$ is a polynomial of degree $(n - k)$ and it is also a factor of $X^n + 1$, then $g(X)$ is the generator polynomial of an (n, k) cyclic code. Equivalently, we may state that if $h(X)$ is a polynomial of degree k and it is also a factor of $X^n + 1$, then $h(X)$ is the parity-check polynomial of an (n, k) cyclic code.

A final comment is in order. Any factor of $X^n + 1$ with degree $(n - k)$, the number of parity bits, can be used as a generator polynomial. For large values of n, the polynomial $X^n + 1$ may have many factors of degree $n - k$. Some of these polynomial factors generate

good cyclic codes, whereas some of them generate bad cyclic codes. The issue of how to select generator polynomials that produce good cyclic codes is very difficult to resolve. Indeed, coding theorists have expended much effort in the search for good cyclic codes.

Generator and Parity-Check Matrices

Given the generator polynomial $g(X)$ of an (n, k) cyclic code, we may construct the generator matrix **G** of the code by noting that the k polynomials $g(X), Xg(X), \ldots, X^{k-1}g(X)$ span the code. Hence, the n-tuples corresponding to these polynomials may be used as rows of the k-by-n generator matrix **G**.

However, the construction of the parity-check matrix **H** of the cyclic code from the parity-check polynomial $h(X)$ requires special attention, as described here. Multiplying Equation (10.42) by $a(x)$ and then using Equation (10.35), we obtain

$$c(X)h(X) = a(X) + X^n a(X) \tag{10.43}$$

The polynomials $c(X)$ and $h(X)$ are themselves defined by Equations (10.27) and (10.40), respectively, which means that their product on the left-hand side of Equation (10.43) contains terms with powers extending up to $n + k - 1$. On the other hand, the polynomial $a(X)$ has degree $k - 1$ or less, the implication of which is that the powers of $X^k, X^{k+1}, \ldots, X^{n-1}$ do *not* appear in the polynomial on the right-hand side of Equation (10.43). Thus, setting the coefficients of $X^k, X^{k-1}, \ldots, X^{n-1}$ in the expansion of the product polynomial $c(X)h(X)$ equal to zero, we obtain the following set of $n - k$ equations:

$$\sum_{i=j}^{j+k} c_i h_{k+j-i} = 0 \qquad \text{for } 0 \le j \le n - k - 1 \tag{10.44}$$

Comparing Equation (10.44) with the corresponding relation of Equation (10.16), we may make the following important observation: The coefficients of the parity-check polynomial $h(X)$ involved in the polynomial multiplication described in Equation (10.44) are arranged in *reversed* order with respect to the coefficients of the parity-check matrix **H** involved in forming the inner product of vectors described in Equation (10.16). This observation suggests that we define the *reciprocal of the parity-check polynomial* as follows:

$$\begin{aligned} X^k h(X^{-1}) &= X^k\left(1 + \sum_{i=1}^{k-1} h_i X^{-i} + X^{-k}\right) \\ &= 1 + \sum_{i=1}^{k-1} h_{k-i} X^i + X^k \end{aligned} \tag{10.45}$$

which is also a factor of $X^n + 1$. The n-tuples pertaining to the $(n - k)$ polynomials $X^k h(X^{-1}), X^{k+1}h(X^{-1}), \ldots, X^{n-1}h(X^{-1})$ may now be used in rows of the $(n - k)$-by-n parity-check matrix **H**.

In general, the generator matrix **G** and the parity-check matrix **H** constructed in the manner described here are not in their systematic forms. They can be put into their systematic forms by performing simple operations on their respective rows, as illustrated in Example 10.3.

Encoder for Cyclic Codes

Earlier we showed that the encoding procedure for an (n, k) cyclic code in systematic form involves three steps: (1) multiplication of the message polynomial $m(X)$ by X^{n-k}, (2) di-

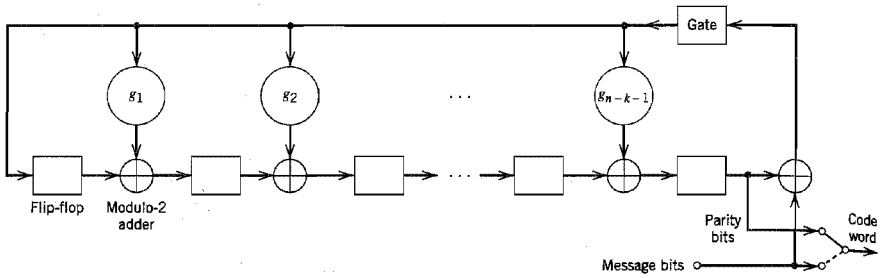


FIGURE 10.8 Encoder for an (n, k) cyclic code.

vision of $X^{n-k}m(X)$ by the generator polynomial $g(X)$ to obtain the remainder $b(X)$, and (3) addition of $b(X)$ to $X^{n-k}m(X)$ to form the desired code polynomial. These three steps can be implemented by means of the encoder shown in Figure 10.8, consisting of a *linear feedback shift register* with $(n - k)$ stages.

The boxes in Figure 10.8 represent *flip-flops*, or *unit-delay elements*. The flip-flop is a device that resides in one of two possible states denoted by 0 and 1. An *external clock* (not shown in Figure 10.8) controls the operation of all the flip-flops. Every time the clock ticks, the contents of the flip-flops (initially set to the state 0) are shifted out in the direction of the arrows. In addition to the flip-flops, the encoder of Figure 10.8 includes a second set of logic elements, namely, *adders*, which compute the modulo-2 sums of their respective inputs. Finally, the *multipliers* multiply their respective inputs by the associated coefficients. In particular, if the coefficient $g_i = 1$, the multiplier is just a direct "connection." If, on the other hand, the coefficient $g_i = 0$, the multiplier is "no connection."

The operation of the encoder shown in Figure 10.8 proceeds as follows:

1. The gate is switched on. Hence, the k message bits are shifted into the channel. As soon as the k message bits have entered the shift register, the resulting $(n - k)$ bits in the register form the parity bits [recall that the parity bits are the same as the coefficients of the remainder $b(X)$].
2. The gate is switched off, thereby breaking the feedback connections.
3. The contents of the shift register are read out into the channel.

■ CALCULATION OF THE SYNDROME

Suppose the code word $(c_0, c_1, \ldots, c_{n-1})$ is transmitted over a noisy channel, resulting in the received word $(r_0, r_1, \ldots, r_{n-1})$. From Section 10.3, we recall that the first step in the decoding of a linear block code is to calculate the syndrome for the received word. If the syndrome is zero, there are no transmission errors in the received word. If, on the other hand, the syndrome is nonzero, the received word contains transmission errors that require correction.

In the case of a cyclic code in systematic form, the syndrome can be calculated easily. Let the received word be represented by a polynomial of degree $n - 1$ or less, as shown by

$$r(X) = r_0 + r_1X + \cdots + r_{n-1}X^{n-1} \tag{10.46}$$

Let $q(X)$ denote the quotient and $s(X)$ denote the remainder, which are the results of dividing $r(X)$ by the generator polynomial $g(X)$. We may therefore express $r(X)$ as follows:

$$r(X) = q(X)g(X) + s(X) \tag{10.47}$$

The remainder $s(X)$ is a polynomial of degree $n - k - 1$ or less, which is the result of interest. It is called the *syndrome polynomial* because its coefficients make up the $(n - k)$-by-1 syndrome s.

Figure 10.9 shows a *syndrome calculator* that is identical to the encoder of Figure 10.8 except for the fact that the received bits are fed into the $(n - k)$ stages of the feedback shift register from the left. As soon as all the received bits have been shifted into the shift register, its contents define the syndrome s.

The syndrome polynomial $s(X)$ has the following useful properties that follow from the definition given in Equation (10.47).

1. *The syndrome of a received word polynomial is also the syndrome of the corresponding error polynomial.*

Given that a cyclic code with polynomial $c(X)$ is sent over a noisy channel, the received word polynomial is defined by

$$r(X) = c(X) + e(X) \tag{10.48}$$

where $e(X)$ is the *error polynomial*. Equivalently, we may write

$$e(X) = r(X) + c(X) \tag{10.49}$$

Hence, substituting Equations (10.35) and (10.47) into (10.49), we get

$$e(X) = u(X)g(X) + s(X) \tag{10.50}$$

where the quotient is $u(X) = a(X) + q(X)$. Equation (10.50) shows that $s(X)$ is also the syndrome of the error polynomial $e(X)$. The implication of this property is that when the syndrome polynomial $s(X)$ is nonzero, the presence of transmission errors in the received word is detected.

2. *Let $s(X)$ be the syndrome of a received word polynomial $r(X)$. Then, the syndrome of $Xr(X)$, a cyclic shift of $r(X)$, is $Xs(X)$.*

Applying a cyclic shift to both sides of Equation (10.47), we get

$$Xr(X) = Xq(X)g(X) + Xs(X) \tag{10.51}$$

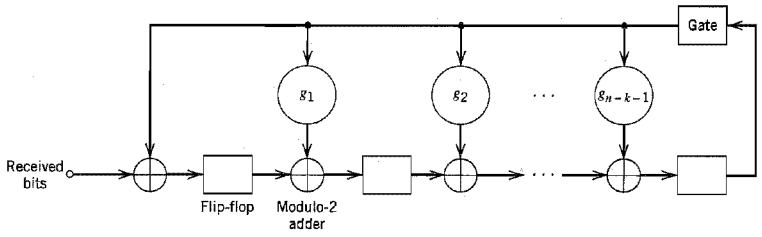


FIGURE 10.9 Syndrome calculator for (n, k) cyclic code.

from which we readily see that $Xs(X)$ is the remainder of the division of $Xr(X)$ by $g(X)$. Hence, the syndrome of $Xr(X)$ is $Xs(X)$ as stated. We may generalize this result by stating that if $s(X)$ is the syndrome of $r(X)$, then $X^i s(X)$ is the syndrome of $X^i r(X)$.

3. *The syndrome polynomial $s(X)$ is identical to the error polynomial $e(X)$, assuming that the errors are confined to the $(n - k)$ parity-check bits of the received word polynomial $r(X)$.*

The assumption made here is another way of saying that the degree of the error polynomial $e(X)$ is less than or equal to $(n - k - 1)$. Since the generator polynomial $g(X)$ is of degree $(n - k)$, by definition, it follows that Equation (10.50) can only be satisfied if the quotient $u(X)$ is zero. In other words, the error polynomial $e(X)$ and the syndrome polynomial $s(X)$ are one and the same. The implication of Property 3 is that, under the aforementioned conditions, error correction can be accomplished simply by adding the syndrome polynomial $s(X)$ to the received word polynomial $r(X)$.

▶ EXAMPLE 10.3 Hamming Codes Revisited

To illustrate the issues relating to the polynomial representation of cyclic codes, we consider the generation of a (7, 4) cyclic code. With the block length $n = 7$, we start by factorizing $X^7 + 1$ into three *irreducible polynomials*:

$$X^7 + 1 = (1 + X)(1 + X^2 + X^3)(1 + X + X^3)$$

By an "irreducible polynomial" we mean a polynomial that cannot be factored using only polynomials with coefficients from the binary field. An irreducible polynomial of degree m is said to be *primitive* if the smallest positive integer n for which the polynomial divides $X^n + 1$ is $n = 2^m - 1$. For the example at hand, the two polynomials $(1 + X^2 + X^3)$ and $(1 + X + X^3)$ are primitive. Let us take

$$g(X) = 1 + X + X^3$$

as the generator polynomial, whose degree equals the number of parity bits. This means that the parity-check polynomial is given by

$$\begin{aligned} h(X) &= (1 + X)(1 + X^2 + X^3) \\ &= 1 + X + X^2 + X^4 \end{aligned}$$

whose degree equals the number of message bits $k = 4$.

Next, we illustrate the procedure for the construction of a code word by using this generator polynomial to encode the message sequence 1001. The corresponding message polynomial is given by

$$m(X) = 1 + X^3$$

Hence, multiplying $m(X)$ by $X^{n-k} = X^3$, we get

$$X^{n-k}m(X) = X^3 + X^6$$

The second step is to divide $X^{n-k}m(X)$ by $g(X)$, the details of which (for the example at hand) are given below:

$$\begin{array}{r} X^3 + X \\ X^3 + X + 1 \overline{)\, X^6 \qquad + X^3} \\ \underline{X^6 \quad + X^4 + X^3} \\ X^4 \\ \underline{X^4 \qquad + X^2 + X} \\ X^2 + X \end{array}$$

Note that in this long division we have treated subtraction the same as addition, since we are operating in modulo-2 arithmetic. We may thus write

$$\frac{X^3 + X^6}{1 + X + X^3} = X + X^3 + \frac{X + X^2}{1 + X + X^3}$$

That is, the quotient $a(X)$ and remainder $b(X)$ are as follows, respectively:

$$a(X) = X + X^3$$
$$b(X) = X + X^2$$

Hence, from Equation (10.38) we find that the desired code polynomial is

$$\begin{aligned} c(X) &= b(X) + X^{n-k}m(X) \\ &= X + X^2 + X^3 + X^6 \end{aligned}$$

The code word is therefore 0111001. The four right-most bits, 1001, are the specified message bits. The three left-most bits, 011, are the parity-check bits. The code word thus generated is exactly the same as the corresponding one shown in Table 10.1 for a (7, 4) Hamming code.

We may generalize this result by stating that *any cyclic code generated by a primitive polynomial is a Hamming code of minimum distance 3.*

We next show that the generator polynomial $g(X)$ and the parity-check polynomial $h(X)$ uniquely specify the generator matrix **G** and the parity-check matrix **H**, respectively.

To construct the 4-by-7 generator matrix **G**, we start with four polynomials represented by $g(X)$ and three cyclic-shifted versions of it, as shown by

$$g(X) = 1 + X + X^3$$
$$Xg(X) = X + X^2 + X^4$$
$$X^2g(X) = X^2 + X^3 + X^5$$
$$X^3g(X) = X^3 + X^4 + X^6$$

The polynomials $g(X)$, $Xg(X)$, $X^2g(X)$, and $X^3g(X)$ represent code polynomials in the (7, 4) Hamming code. If the coefficients of these polynomials are used as the elements of the rows of a 4-by-7 matrix, we get the following generator matrix:

$$\mathbf{G}' = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Clearly, the generator matrix **G**′ so constructed is not in systematic form. We can put it into a systematic form by adding the first row to the third row, and adding the sum of the first two rows to the fourth row. These manipulations result in the desired generator matrix:

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

which is exactly the same as that in Example 10.2.

We next show how to construct the 3-by-7 parity-check matrix **H** from the parity-check polynomial $h(X)$. To do this, we first take the *reciprocal* of $h(X)$, namely, $X^4h(X^{-1})$. For the problem at hand, we form three polynomials represented by $X^4h(X^{-1})$ and two shifted versions of it, as shown by

$$X^4h(X^{-1}) = 1 + X^2 + X^3 + X^4$$
$$X^5h(X^{-1}) = X + X^3 + X^4 + X^5$$
$$X^6h(X^{-1}) = X^2 + X^4 + X^5 + X^6$$

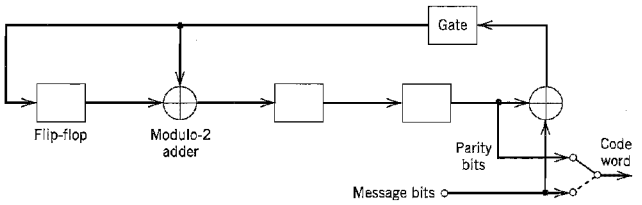


FIGURE 10.10 Encoder for the (7, 4) cyclic code generated by $g(X) = 1 + X + X^3$.

Using the coefficients of these three polynomials as the elements of the rows of the 3-by-7 parity-check matrix, we get

$$\mathbf{H}' = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

Here again we see that the matrix $\mathbf{H}'$ is not in systematic form. To put it into a systematic form, we add the third row to the first row to obtain

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

which is exactly the same as that of Example 10.2.

Figure 10.10 shows the encoder for the (7, 4) cyclic Hamming code generated by the polynomial $g(X) = 1 + X + X^3$. To illustrate the operation of this encoder, consider the message sequence (1001). The contents of the shift register are modified by the incoming message bits as in Table 10.3. After four shifts, the contents of the shift register, and therefore the parity bits, are (011). Accordingly, appending these parity bits to the message bits (1001), we get the code word (0111001); this result is exactly the same as that determined earlier in the example.

Figure 10.11 shows the corresponding syndrome calculator for the (7, 4) Hamming code. Let the transmitted code word be (0111001) and the received word be (0110001); that is, the middle bit is in error. As the received bits are fed into the shift register, initially set to zero, its contents are modified as in Table 10.4. At the end of the seventh shift, the syndrome is identified from the contents of the shift register as 110. Since the syndrome is nonzero, the received word is in error. Moreover, from Table 10.2, we see that the error pattern corresponding to this syndrome is 0001000. This indicates that the error is in the middle bit of the received word, which is indeed the case. ◀

TABLE 10.3 ***Contents of the shift register in the encoder of Figure 10.10 for message sequence (1001)***

Shift	*Input*	*Register Contents*
		0 0 0 (initial state)
1	1	1 1 0
2	0	0 1 1
3	0	1 1 1
4	1	0 1 1

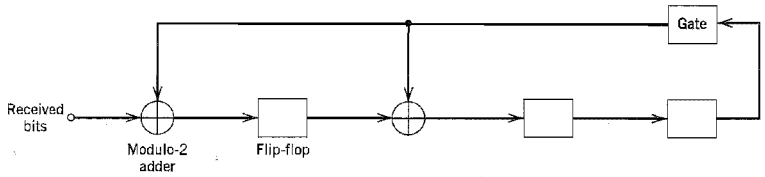


FIGURE 10.11 Syndrome calculator for the (7, 4) cyclic code generated by the polynomial $g(X) = 1 + X + X^3$.

▶ EXAMPLE 10.4 Maximal-Length Codes

For any positive integer $m \geq 3$, there exists a *maximal-length code* with the following parameters:

Block length:	$n = 2^m - 1$
Number of message bits:	$k = m$
Minimum distance:	$d_{\min} = 2^{m-1}$

Maximal-length codes are generated by polynomials of the form

$$g(X) = \frac{1 + X^n}{h(X)} \tag{10.52}$$

where $h(X)$ is any primitive polynomial of degree m. Earlier we stated that any cyclic code generated by a primitive polynomial is a Hamming code of minimum distance 3 (see Example 10.3). It follows therefore that maximal-length codes are the *dual* of Hamming codes.

The polynomial $h(X)$ defines the feedback connections of the encoder. The generator polynomial $g(X)$ defines one period of the maximal-length code, assuming that the encoder is in the initial state 00 . . . 01. To illustrate these points, consider the example of a (7, 3) maximal-length code, which is the dual of the (7, 4) Hamming code described in Example 10.3. Thus, choosing

$$h(X) = 1 + X + X^3$$

we find that the generator polynomial of the (7, 3) maximal-length code is

$$g(X) = 1 + X + X^2 + X^4$$

TABLE 10.4 *Contents of the syndrome calculator in Figure 10.11 for the received word 0110001*

Shift	*Input Bit*	*Contents of Shift Register*
		0 0 0 (initial state)
1	1	1 0 0
2	0	0 1 0
3	0	0 0 1
4	0	1 1 0
5	1	1 1 1
6	1	0 0 1
7	0	1 1 0

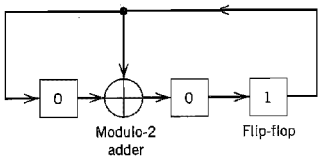


FIGURE 10.12 Encoder for the (7, 3) maximal-length code; the initial state of the encoder is shown in the figure.

Figure 10.12 shows the encoder for the (7, 3) maximal-length code, the feedback connections of which are exactly the same as those shown in Figure 8.2 in Chapter 8. The period of the code is $n = 7$. Thus, assuming that the encoder is in the initial state 001, as indicated in Figure 10.12, we find the output sequence is described by

$$\underbrace{1\ 0\ 0}_{\text{initial state}} \qquad \underbrace{1\ 1\ 1\ 0\ 1\ 0\ 0}_{g(X) = 1 + X + X^2 + X^4}$$

This result may be readily validated by cycling through the encoder of Figure 10.12.

Note that if we were to choose the other primitive polynomial

$$h(X) = 1 + X^2 + X^3$$

for the (7, 3) maximal-length code, we would simply get the "image" of the code described above, and the output sequence would be "reversed" in time. ◀

■ OTHER CYCLIC CODES

We conclude the discussion of cyclic codes by presenting the characteristics of three other important classes of cyclic codes.

Cyclic Redundancy Check Codes

Cyclic codes are extremely well-suited for *error detection*. We make this statement for two reasons. First, they can be designed to detect many combinations of likely errors. Second, the implementation of both encoding and error-detecting circuits is practical. It is for these reasons that many of the error-detecting codes used in practice are of the cyclic-code type. A cyclic code used for error-detection is referred to as *cyclic redundancy check (CRC) code.*

We define an *error burst* of length B in an n-bit received word as a contiguous sequence of B bits in which the first and last bits or any number of intermediate bits are received in error. Binary (n, k) CRC codes are capable of detecting the following error patterns:

1. All error bursts of length $n - k$ or less.
2. A fraction of error bursts of length equal to $n - k + 1$; the fraction equals $1 - 2^{-(n-k-1)}$.
3. A fraction of error bursts of length greater than $n - k + 1$; the fraction equals $1 - 2^{-(n-k-1)}$.
4. All combinations of $d_{\min} - 1$ (or fewer) errors.
5. All error patterns with an odd number of errors if the generator polynomial $g(X)$ for the code has an even number of nonzero coefficients.

TABLE 10.5 *CRC codes*

Code	*Generator Polynomial, g(X)*	n − k
CRC-12 code	$1 + X + X^2 + X^3 + X^{11} + X^{12}$	12
CRC-16 code (USA)	$1 + X^2 + X^{15} + X^{16}$	16
CRC-ITU code	$1 + X^5 + X^{12} + X^{16}$	16

Table 10.5 presents the generator polynomials of three CRC codes that have become international standards. All three codes contain $1 + X$ as a prime factor. The CRC-12 code is used for 6-bit characters, and the other two codes are used for 8-bit characters. CRC codes provide a powerful method of error detection for use in automatic-repeat request (ARQ) strategies discussed in Section 10.1, and digital subscriber lines discussed in Chapter 4.

Bose–Chaudhuri–Hocquenghem (BCH) Codes[5]

One of the most important and powerful classes of linear-block codes are *BCH codes*, which are cyclic codes with a wide variety of parameters. The most common binary BCH codes, known as *primitive BCH codes*, are characterized for any positive integers m (equal to or greater than 3) and t [less than $(2^m - 1)/2$] by the following parameters:

Block length: $n = 2^m - 1$

Number of message bits: $k \geq n - mt$

Minimum distance: $d_{\min} \geq 2t + 1$

Each BCH code is a *t-error correcting code* in that it can detect and correct up to t random errors per code word. The Hamming single-error correcting codes can be described as BCH codes. The BCH codes offer flexibility in the choice of code parameters, namely, block length and code rate. Furthermore, for block lengths of a few hundred bits or less, the BCH codes are among the best known codes of the same block length and code rate.

A detailed treatment of the construction of BCH codes is beyond the scope of our present discussion. To provide a feel for their capability, we present in Table 10.6, the code parameters and generator polynomials for binary block BCH codes of length up to $2^5 - 1$. For example, suppose we wish to construct the generator polynomial for (15, 7)

TABLE 10.6 *Binary BCH codes of length up to $2^5 - 1$*

n	k	t	*Generator Polynomial*								
7	4	1								1	011
15	11	1								10	011
15	7	2							111	010	001
15	5	3						10	100	110	111
31	26	1								100	101
31	21	2						11	101	101	001
31	16	3				1	000	111	110	101	111
31	11	5			101	100	010	011	011	010	101
31	6	7	11	001	011	011	110	101	000	100	111

Notation: n = block length
k = number of message bits
t = maximum number of detectable errors

The high-order coefficients of the generator polynomial $g(X)$ are at the left.

BCH code. From Table 10.6 we have (111 010 001) for the coefficients of the generator polynomial; hence, we write

$$g(X) = X^8 + X^7 + X^6 + X^4 + 1$$

Reed–Solomon Codes[6]

The *Reed–Solomon codes* are an important subclass of *nonbinary* BCH codes; they are often abbreviated as RS codes. The encoder for an RS code differs from a binary encoder in that it operates on multiple bits rather than individual bits. Specifically, an RS (n, k) code is used to encode m-bit symbols into blocks consisting of $n = 2^m - 1$ symbols, that is, $m(2^m - 1)$ bits, where $m \geq 1$. Thus, the encoding algorithm expands a block of k symbols to n symbols by adding $n - k$ redundant symbols. When m is an integer power of two, the m-bit symbols are called *bytes*. A popular value of m is 8; indeed, 8-bit RS codes are extremely powerful.

A t-error-correcting RS code has the following parameters:

Block length:	$n = 2^m - 1$ symbols
Message size:	k symbols
Parity-check size:	$n - k = 2t$ symbols
Minimum distance:	$d_{\min} = 2t + 1$ symbols

The block length of the RS code is one less than the size of a code symbol, and the minimum distance is one greater than the number of parity-check symbols. The RS codes make highly efficient use of redundancy, and block lengths and symbol sizes can be adjusted readily to accommodate a wide range of message sizes. Moreover, the RS codes provide a wide range of code rates that can be chosen to optimize performance. Finally, efficient decoding techniques are available for use with RS codes, which is one more reason for their wide application (e.g., compact disc digital audio systems).

10.5 *Convolutional Codes*[7]

In block coding, the encoder accepts a k-bit message block and generates an n-bit code word. Thus, code words are produced on a block-by-block basis. Clearly, provision must be made in the encoder to buffer an entire message block before generating the associated code word. There are applications, however, where the message bits come in *serially* rather than in large blocks, in which case the use of a buffer may be undesirable. In such situations, the use of *convolutional coding* may be the preferred method. A convolutional coder generates redundant bits by using *modulo-2 convolutions*, hence the name.

The encoder of a binary convolutional code with rate $1/n$, measured in bits per symbol, may be viewed as a *finite-state machine* that consists of an M-stage shift register with prescribed connections to n modulo-2 adders, and a multiplexer that serializes the outputs of the adders. An L-bit message sequence produces a coded output sequence of length $n(L + M)$ bits. The *code rate* is therefore given by

$$r = \frac{L}{n(L + M)} \quad \text{bits/symbol} \tag{10.53}$$

Typically, we have $L \gg M$. Hence, the code rate simplifies to

$$r \simeq \frac{1}{n} \quad \text{bits/symbol} \tag{10.54}$$

The *constraint length* of a convolutional code, expressed in terms of message bits, is defined as the number of shifts over which a single message bit can influence the encoder output. In an encoder with an M-stage shift register, the *memory* of the encoder equals M message bits, and $K = M + 1$ shifts are required for a message bit to enter the shift register and finally come out. Hence, the constraint length of the encoder is K.

Figure 10.13*a* shows a convolutional encoder with $n = 2$ and $K = 3$. Hence, the code rate of this encoder is 1/2. The encoder of Figure 10.13*a* operates on the incoming message sequence, one bit at a time.

We may generate a binary convolutional code with rate k/n by using k separate shift registers with prescribed connections to n modulo-2 adders, an input multiplexer and

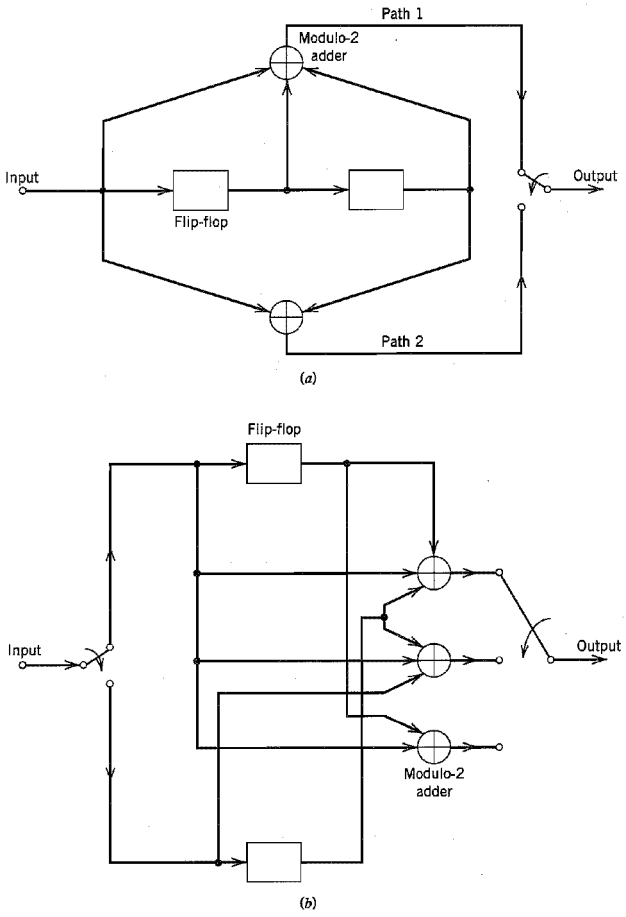


FIGURE 10.13 (*a*) Constraint length-3, rate-$\frac{1}{2}$ convolutional encoder. (*b*) Constraint length-2, rate-$\frac{2}{3}$ convolutional encoder.

an output multiplexer. An example of such an encoder is shown in Figure 10.13*b*, where $k = 2$, $n = 3$, and the two shift registers have $K = 2$ each. The code rate is 2/3. In this second example, the encoder processes the incoming message sequence two bits at a time.

The convolutional codes generated by the encoders of Figure 10.13 are *nonsystematic* codes. Unlike block coding, the use of nonsystematic codes is ordinarily preferred over systematic codes in convolutional coding.

Each path connecting the output to the input of a convolutional encoder may be characterized in terms of its *impulse response*, defined as the response of that path to a symbol 1 applied to its input, with each flip-flop in the encoder set initially in the zero state. Equivalently, we may characterize each path in terms of a *generator polynomial*, defined as the *unit-delay transform* of the impulse response. To be specific, let the *generator sequence* $(g_0^{(i)}, g_1^{(i)}, g_2^{(i)}, \ldots, g_M^{(i)})$ denote the impulse response of the ith path, where the coefficients $g_0^{(i)}, g_1^{(i)}, g_2^{(i)}, \ldots, g_M^{(i)}$ equal 0 or 1. Correspondingly, the *generator polynomial* of the ith path is defined by

$$g^{(i)}(D) = g_0^{(i)} + g_1^{(i)}D + g_2^{(i)}D^2 + \cdots + g_M^{(i)}D^M \tag{10.55}$$

where D denotes the unit-delay variable. The complete convolutional encoder is described by the set of generator polynomials $\{g^{(1)}(D), g^{(2)}(D), \ldots, g^{(n)}(D)\}$. Traditionally, different variables are used for the description of convolutional and cyclic codes, with D being commonly used for convolutional codes and X for cyclic codes.

▶ EXAMPLE 10.5

Consider the convolutional encoder of Figure 10.13*a*, which has two paths numbered 1 and 2 for convenience of reference. The impulse response of path 1 (i.e., upper path) is (1, 1, 1). Hence, the cortesponding generator polynomial is given by

$$g^{(1)}(D) = 1 + D + D^2$$

The impulse response of path 2 (i.e., lower path) is (1, 0, 1). Hence, the corresponding generator polynomial is given by

$$g^{(2)}(D) = 1 + D^2$$

For the message sequence (10011), say, we have the polynomial representation

$$m(D) = 1 + D^3 + D^4$$

As with Fourier transformation, convolution in the time domain is transformed into multiplication in the D-domain. Hence, the output polynomial of path 1 is given by

$$\begin{aligned} c^{(1)}(D) &= g^{(1)}(D)m(D) \\ &= (1 + D + D^2)(1 + D^3 + D^4) \\ &= 1 + D + D^2 + D^3 + D^6 \end{aligned}$$

From this we immediately deduce that the output sequence of path 1 is (1111001). Similarly, the output polynomial of path 2 is given by

$$\begin{aligned} c^{(2)}(D) &= g^{(2)}(D)m(D) \\ &= (1 + D^2)(1 + D^3 + D^4) \\ &= 1 + D^2 + D^3 + D^4 + D^5 + D^6 \end{aligned}$$

The output sequence of path 2 is therefore (1011111). Finally, multiplexing the two output sequences of paths 1 and 2, we get the encoded sequence

$$\mathbf{c} = (11, 10, 11, 11, 01, 01, 11)$$

Note that the message sequence of length $L = 5$ bits produces an encoded sequence of length $n(L + K - 1) = 14$ bits. Note also that for the shift register to be restored to its zero initial state, a terminating sequence of $K - 1 = 2$ zeros is appended to the last input bit of the message sequence. The terminating sequence of $K - 1$ zeros is called the *tail of the message*. ◀

Code Tree, Trellis, and State Diagram

Traditionally, the structural properties of a convolutional encoder are portrayed in graphical form by using any one of three equivalent diagrams: code tree, trellis, and state diagram. We will use the convolutional encoder of Figure 10.13*a* as a running example to illustrate the insights that each one of these three diagrams can provide.

We begin the discussion with the *code tree* of Figure 10.14. Each branch of the tree represents an input symbol, with the corresponding pair of output binary symbols indicated on the branch. The convention used to distinguish the input binary symbols 0 and 1 is as follows. An input 0 specifies the upper branch of a bifurcation, whereas input 1 specifies the lower branch. A specific *path* in the tree is traced from left to right in accordance with the input (message) sequence. The corresponding coded symbols on the branches of that path constitute the input (message) sequence. Consider, for example, the message sequence (10011) applied to the input of the encoder of Figure 10.13*a*. Following the procedure just described, we find that the corresponding encoded sequence is (11, 10, 11, 11, 01), which agrees with the first 5 pairs of bits in the encoded sequence $\{c_i\}$ derived in Example 10.5.

From the diagram of Figure 10.14, we observe that the tree becomes *repetitive* after the first three branches. Indeed, beyond the third branch, the two nodes labeled *a* are identical, and so are all the other node pairs that are identically labeled. We may establish this repetitive property of the tree by examining the associated encoder of Figure 10.13*a*. The encoder has memory $M = K - 1 = 2$ message bits. Hence, when the third message bit enters the encoder, the first message bit is shifted out of the register. Consequently, after the third branch, the message sequences ($100\, m_3 m_4 \ldots$) and ($000\, m_3 m_4 \ldots$) generate the same code symbols, and the pair of nodes labeled *a* may be joined together. The same reasoning applies to other nodes. Accordingly, we may collapse the code tree of Figure 10.14 into the new form shown in Figure 10.15, which is called a *trellis*.[8] It is so called since a trellis is a treelike structure with remerging branches. The convention used in Figure 10.15 to distinguish between input symbols 0 and 1 is as follows. A code branch produced by an input 0 is drawn as a solid line, whereas a code branch produced by an input 1 is drawn as a dashed line. As before, each input (message) sequence corresponds to a specific path through the trellis. For example, we readily see from Figure 10.15 that the message sequence (10011) produces the encoded output sequence (11, 10, 11, 11, 01), which agrees with our previous result.

A trellis is more instructive than a tree in that it brings out explicitly the fact that the associated convolutional encoder is a *finite-state machine*. We define the *state* of a convolutional encoder of rate $1/n$ as the $(K - 1)$ message bits stored in the encoder's shift register. At time j, the portion of the message sequence containing the most recent K bits is written as $(m_{j-K+1}, \ldots, m_{j-1}, m_j)$, where m_j is the *current* bit. The $(K - 1)$-bit state of the encoder at time j is therefore written simply as $(m_{j-1}, \ldots, m_{j-K+2}, m_{j-K+1})$. In the